



TRABAJO DE FINAL DE GRADO

GRADO EN INTELIGENCIA ROBÓTICA

Estudio e implementación del control de péndulo invertido sobre carro en un sistema de tiempo real

Estudiante:
Miguel Porcar Vicent

Tutor:
Ignacio Peñarrocha Alós

Fecha de presentación: Junio de 2026
Curso académico: 2025/2026

Agradecimientos:

Palabras clave:

CASTELLÓN, a XX de XXXXXXX de 2025

Todos los derechos reservados

ÍNDICE GENERAL

1	Introducción	5
1.1	Punto de partida	6
1.2	Motivaciones	8
1.3	Planificación	8
1.3.1	Objetivos	8
1.3.2	Tareas	9
1.3.3	Estimación de tiempos	10
1.4	Estimación de costes	10
2	Análisis	11
2.1	Requisitos funcionales	12
2.2	Requisitos no funcionales	13
2.3	Diseño del sistema	14
2.3.1	Introducción al modelado dinámico	14
2.3.2	Modelo dinámico del sistema	15
2.3.3	Estrategia de control Swing-up	17
2.3.4	Modelado en el espacio de estados	20
2.3.5	Control por realimentación del estado	22
2.3.6	Lógica de conmutación	23
2.4	Arquitectura del sistema	24
2.4.1	Arquitectura de la simulación	24
2.4.2	Arquitectura del sistema real	25
2.4.3	Especificación de componentes de hardware	26

3	Desarrollo	31
3.1	Trabajo de simulación	32
3.1.1	Modelado del sistema en Simulink	32
3.1.2	Algoritmo de control en Simulink	33
3.1.3	Comunicación con ROS y Visualización del Gemelo Digital	36
3.1.4	Simulaciones Recursivas	37
3.2	Programación del microcontrolador	39
3.2.1	Módulo eQEP (Lectura del Encoder)	42
3.2.2	Módulo ePWM1 (Actuación del Motor)	43
3.2.3	Protocolo de comunicación serie y telemetría (SCI)	47
3.2.4	Rutina de interrupción periódica	47
3.3	Identificación de parámetros	47
3.4	Implementación del algoritmo de control	47
4	Resultados	49
4.1	Resultados en simulación	49
4.2	Resultados en sistema físico	49
5	Conclusiones y posible trabajo futuro	51
	Bibliografía	53
6	Apéndices	55
A	Desarrollo del Lagrangiano	55
B	Dinámica en MATLAB Functions	56
C	Código Control de Energía	58
D	Configuración Conexión MATLAB ROS	59
E	Desarrollo Dinámico y Programación del Bucle de Simulación	61
F	Tabla de Multiplexación Completa del TMS32F280049C	62
G	Registros del módulo eQEP	64
H	Registros del módulo ePWM	68

1. Introducción

El objeto de este proyecto es el estudio y control de un péndulo invertido sobre un carro móvil. Aunque a primera vista pueda parecer un sistema mecánico simple, en el ámbito de la **Robótica** este dispositivo constituye uno de los problemas fundamentales de control y equilibrio dinámico.

La relevancia de este sistema reside en su transferencia directa a tecnologías críticas de la robótica moderna. El reto de mantener una barra inestable en equilibrio vertical es, en esencia, el mismo principio físico que permite a un **robot humanoide** caminar sin desplomarse, a un cohete mantenerse estable durante el despegue o a vehículos de transporte personal (como los *Segways*) desplazarse de forma autónoma. Controlar este sistema implica dominar la capacidad de un robot para reaccionar ante la gravedad y las perturbaciones externas en milisegundos.

En ingeniería, este reto se utiliza para evaluar la eficacia de un sistema de control. El objetivo es que un microcontrolador (cerebro) interprete la inclinación de la barra mediante sensores (ojos) y actúe sobre un motor (músculos) con una velocidad y precisión tales que desafíen la inestabilidad natural del sistema.

Finalmente, en el marco de la robótica, este proyecto se sitúa en la denominada **capa reactiva**. Mientras que gran parte de la robótica actual se enfoca en procesos cognitivos de alto nivel —como la navegación mediante SLAM—, este sistema aborda el control a bajo nivel, equiparable a los **reflejos motores** de un ser vivo. No se trata de "pensar" una trayectoria a largo plazo, sino de reaccionar en milisegundos para corregir desviaciones imperceptibles. Esta capacidad de respuesta inmediata es la que garantiza que el robot sea capaz de interactuar con el mundo físico de forma segura, sirviendo de base fundamental sobre la que se construyen las capas de inteligencia superior.

1.1 Punto de partida

El trabajo se realiza sobre un equipo educativo clásico: el **Digital Pendulum 33-005** de la marca *Feedback Instruments Ltd.* Este sistema se divide en dos partes: el controlador 33-201 (la caja con la electrónica) y la unidad mecánica 33-200 (el raíl con el carro y la barra).

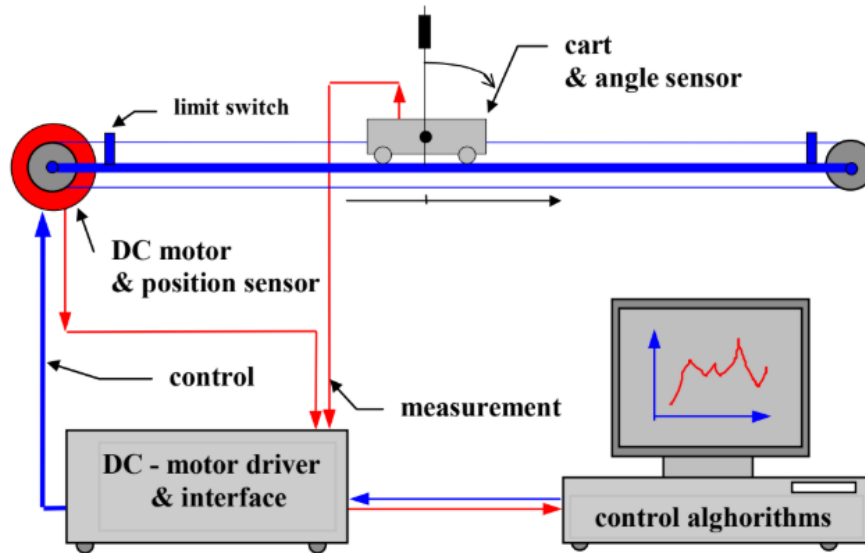


Figura 1.1: Digital Pendulum 33-005, descripción esquemática [1]

En la Figura 1.1 se detalla la disposición de la unidad mecánica 33-200. El sistema consta de un raíl sobre el que se desplaza horizontalmente un carro metálico. Este movimiento es accionado por un motor de corriente continua situado en un extremo, que transmite la fuerza mediante una correa dentada. En los extremos del raíl se sitúan los finales de carrera, que actúan como interruptores de seguridad para detener el motor antes de que el carro colisione con la estructura. Sobre el carro se monta el eje del péndulo, cuya rotación libre de 360 grados es captada por un sensor óptico (encoder) situado en su base.

Este equipo lleva en los laboratorios de la universidad más de 25 años. Debido a este largo periodo de tiempo, el punto de partida del proyecto se caracteriza por una obsolescencia tecnológica total:

- **Electrónica antigua:** Según la documentación del fabricante, este sistema fue diseñado para operar con **MATLAB 5** y **Windows 95**, utilizando protocolos de comunicación y tarjetas de expansión (bus ISA/PCI) que ya no existen en los ordenadores actuales [2]. Esto lo convertía en una "caja negra" cerrada e incompatible con las herramientas de programación modernas, lo que justificaba su sustitución completa por un sistema de control digital actual.

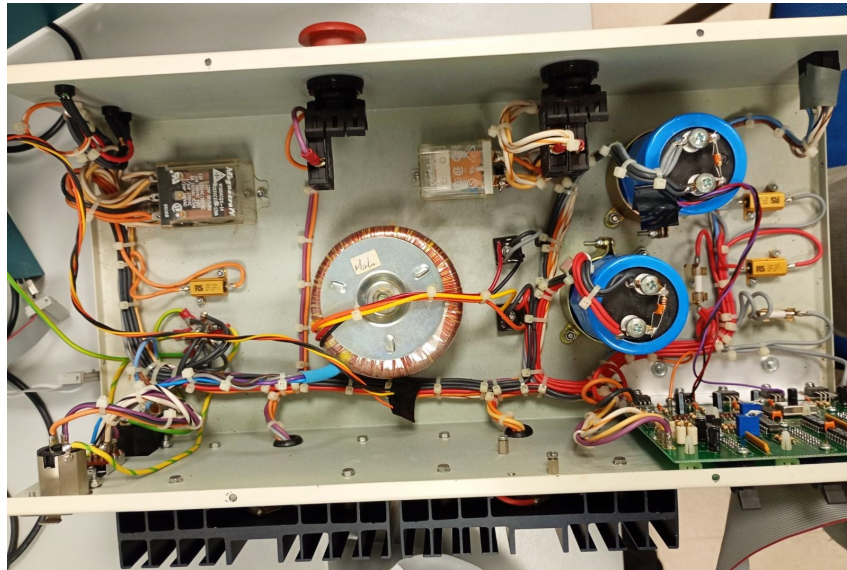
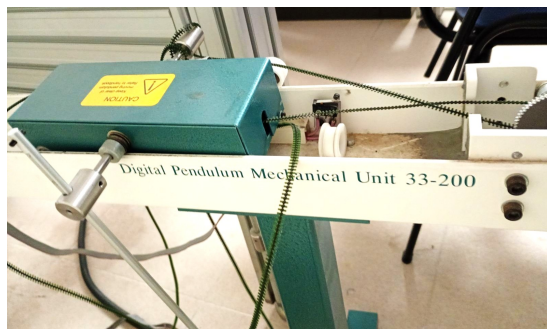
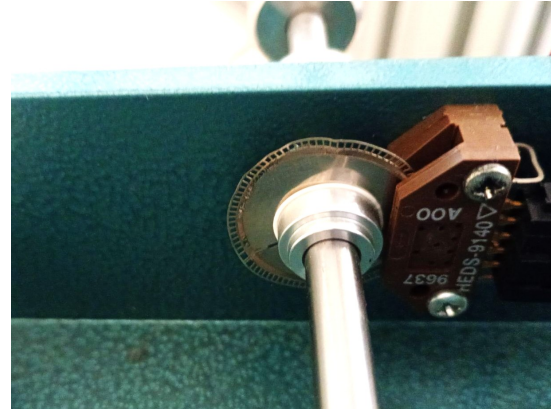


Figura 1.2: Controlador digital original

- **Falta de mantenimiento:** La unidad mecánica presentaba suciedad acumulada en los raíles y grasa reseca en los engranajes, lo que impedía que el carro se moviera con suavidad, complicando enormemente la tarea de equilibrio. Además de tener un desgaste notable en la correa dentada (transmisión) y el disco óptico del encoder.



(a) Unidad mecánica con falta de mantenimiento



(b) Disco óptico doblado y con pérdida de pulsos

Figura 1.3: Estado inicial de la unidad mecánica

Luego el punto de partida es el **rescate de la unidad mecánica**, aprovechando aquellos elementos que aún conservaban su utilidad, como el motor. No obstante, ha sido necesario intervenir en otros componentes que, debido a su naturaleza como piezas de desgaste, presentaban falta de mantenimiento. Por otro lado, para solucionar el problema del control digital obsoleto, se decidió **sustituir la electrónica antigua** por tecnología actual (microcontrolador de la familia *C2000 de Texas Instruments* y drivers modernos). Esta elección no solo permite que la unidad vuelva a ser plenamente funcional, sino que además la convierte en una plataforma de **código abierto**, facilitando que futuros usuarios puedan entender, modificar y mejorar el sistema sin las restricciones del hardware propietario original.

1.2 Motivaciones

Las razones que impulsan el desarrollo de este proyecto combinan el interés personal por ciertos conocimientos adquiridos en el grado y la oportunidad de brindar un servicio a la universidad:

- **Personal y Vocacional:** Mi interés principal dentro del *Grado en Inteligencia Robótica* reside en el control automático, la programación de sistemas empotrados en tiempo real y la electrónica. Este proyecto me permite profundizar en la asignatura de *Sistemas Automáticos Avanzados*, enfrentándome al reto de diseñar algoritmos de control a bajo nivel con restricciones críticas de tiempo y hardware que imponen los sistemas físicos reales.
- **Curiosidad Científica:** Me motiva entender el funcionamiento interno de las máquinas. El control automático me brinda la oportunidad perfecta para aplicar de forma práctica la física y las matemáticas que subyacen a los sistemas robóticos capaces de equilibrarse y moverse con precisión.
- **Profesional:** El uso de la familia de microcontroladores *C2000 de Texas Instruments* constituye una motivación clave. Superar las capas de abstracción de *Arduino IDE* para trabajar directamente sobre el registro y el hardware de los *C2000* me permite entender el control de bajo nivel de forma real. Esta capacidad para gestionar sistemas complejos con estándares industriales constituye una ventaja competitiva.
- **Académica:** Durante este último curso, he disfrutado de la *Beca de Colaboración del Ministerio* en el *Departamento de Ingeniería de Sistemas Industriales y Diseño*. Parte de mi labor ha consistido en poner en marcha esta infraestructura, por lo que este TFG es la culminación del trabajo. Dejar una unidad funcional y actualizada que sirva como banco de pruebas para que futuros estudiantes e investigadores puedan seguir evolucionando algoritmos y técnicas de control avanzado en la universidad sería un gran honor.

1.3 Planificación

Dentro de este apartado se aborda de forma lógica y ordenada los objetivos generales del proyecto y las tareas asociadas para lograr su consecución. Además se expone el *Diagrama de Gantt* para tener una visión general de la planificación de tiempos por cada tarea.

1.3.1 Objetivos

- **O1:** Obtención del modelo matemático del sistema de péndulo invertido sobre carro.
- **O2:** Simulación en *Matlab/Simulink* para la validación preliminar de los controladores e integración con *ROS*.
- **O3:** Diseño y evaluación en simulación diferentes estrategias de control.
- **O4:** Caracterización y puesta a punto el hardware de la planta real, incluyendo la etapa de potencia y la lectura de sensores.
- **O5:** Desarrollo el firmware de bajo nivel en el microcontrolador, garantizando el control en tiempo real.
- **O6:** Implementación, ajuste y validación experimental de los algoritmos de control sobre la unidad mecánica real.
- **O7:** Documentación técnica del trabajo realizado y los resultados obtenidos.

1.3.2 Tareas

Cada tarea y subtarea tiene un objetivo asociado para facilitar el desarrollo del proyecto en pequeños retos dentro de un plazo. De este modo la organización es mayor y el enfoque del trabajo es claro.

T1. Modelado matemático y físico (O1)

- T1.1 Obtención de las ecuaciones de movimiento mediante Euler-Lagrange.
- T1.2 Identificación de parámetros cinemáticos y dinámicos (masas, longitudes, inercias).
- T1.3 Caracterización de las no linealidades: fricción estática y dinámica en el raíl y zona muerta del motor.

T2. Entorno de simulación e integración con ROS (O2)

- T2.1 Implementación del modelo dinámico en bloques de *Simulink*.
- T2.2 Configuración del nodo de comunicación entre *Matlab* y el ecosistema *ROS* para el intercambio de datos.
- T2.3 Desarrollo de un entorno visual para la monitorización de la simulación.

T3. Diseño de estrategias de control en simulación (O3)

- T3.1 Diseño del control de posición mediante regulador cuadrático lineal (LQR).
- T3.2 Desarrollo del algoritmo de *Swing-up* basado en el control de energía.

T4. Preparación del hardware y electrónica (O4)

- T4.1 Revisión mecánica y mantenimiento de la unidad *Feedback 33-200* (limpieza, engrasado y ajuste de tensión).
- T4.2 Integración del driver de potencia y diseño del conexionado con los encoders ópticos.
- T4.3 Implementación de sistemas de seguridad físicos (finales de carrera) y lógicos.

T5. Desarrollo del firmware de bajo nivel (O5)

- T5.1 Configuración de periféricos de captura (eQEP) y generación de señales PWM en el microcontrolador.
- T5.2 Programación del bucle de control con una rutina de interrupción periódica.
- T5.3 Implementación del protocolo de comunicación entre el microcontrolador y el ordenador para obtener la telemetría.

T6. Validación experimental en el sistema real (O6)

- T6.1 Ejecución y ajuste fino de los controladores (*tuning*) sobre la planta física.
- T6.2 Comparativa de rendimiento entre los resultados obtenidos en el laboratorio y la simulación previa.

T7. Documentación y cierre (O7)

- T7.1 Elaboración de la memoria técnica del proyecto.
- T7.2 Preparación de la defensa pública y material visual de apoyo.

1.3.3 Estimación de tiempos**1.4 Estimación de costes**

2. Análisis

En este capítulo se realiza un estudio detallado de los pilares que sustentan el desarrollo del sistema de control para el péndulo invertido. El análisis se articula en tres ejes fundamentales:

- **Análisis de requisitos:** Donde se formalizan las capacidades operativas (funcionales) y las restricciones técnicas, de seguridad y rendimiento (no funcionales) que debe satisfacer el sistema.
- **Diseño del sistema:** Se describe la estrategia lógica y matemática adoptada, detallando el modelo dinámico, las leyes de control híbridas y la lógica de conmutación necesaria para el *swing-up* y la estabilización.
- **Arquitectura del sistema:** Se define la infraestructura tanto de simulación como de implementación real, especificando el flujo de datos, la integración con ROS y la selección de los componentes de hardware que garantizan la viabilidad del proyecto.

2.1 Requisitos funcionales

Los requisitos funcionales describen las acciones, cálculos y manipulaciones de datos específicos que el sistema debe realizar para cumplir su propósito. Para el control del péndulo invertido sobre carro se definen mediante sus entradas, comportamiento y salidas:

■ RF1. Modelado y simulación de la dinámica:

- *Entradas:* Parámetros físicos del sistema (masas, inercias, coeficientes de fricción) y señal de control simulada.
- *Comportamiento:* Resolución matemática de las Ecuaciones Diferenciales Ordinarias (EDO) que rigen la dinámica no lineal del mecanismo en el entorno MATLAB/Simulink.
- *Salidas:* Estado cinemático simulado del sistema (posiciones y velocidades articulares).

■ RF2. Visualización y telemetría en ROS:

- *Entradas:* Estado cinemático del sistema (proveniente de la simulación o del hardware real) y archivo de descripción geométrica URDF.
- *Comportamiento:* Traducción y publicación de los datos mediante el puente de comunicación de ROS Toolbox para actualizar el árbol de transformadas (TF) del robot.
- *Salidas:* Visualización 3D actualizada en tiempo real mediante la interfaz RViz (tópico */joint_states*).

■ RF3. Control de inyección de energía (Swing-up):

- *Entradas:* Posición y velocidad angular actual del péndulo $(q, \dot{q})$.
- *Comportamiento:* Cálculo y ejecución de una ley de control basada en energía para balancear el péndulo desde su posición estable ($q = 0$ rad) hasta el punto de linealización del sistema ($q = \pi$ rad).
- *Salidas:* Acción de control (u) .

■ RF4. Control de estabilización (Realimentación del estado):

- *Entradas:* Vector de estado del sistema $(x = [x, \dot{x}, \theta, \dot{\theta}]^T \equiv [q, \dot{q}]^T)$.
- *Comportamiento:* Estabilización del sistema sobre el punto de linealización utilizando control por realimentación del estado.
- *Salidas:* Acción de control (u) .

■ RF5. Adquisición y decodificación de señales:

- *Entradas:* Señales eléctricas de pulsos en cuadratura (A y B) provenientes de los encoders del motor y del péndulo.
- *Comportamiento:* Procesamiento y conteo de los pulsos digitales mediante el hardware específico.
- *Salidas:* Valores digitales discretos de posición y velocidad interpretables por la ley de control.

2.2 Requisitos no funcionales

Los requisitos no funcionales imponen condiciones y restricciones a la implementación del diseño:

- **RNF1. Restricciones de rendimiento (Determinismo y tiempo real):** El bucle de control debe ejecutarse dentro de una rutina de interrupción periódica en un microcontrolador, garantizando un periodo de muestreo estable y un *jitter* despreciable para la viabilidad del control discreto.
- **RNF2. Restricciones de fiabilidad (Robustez):** El sistema debe rechazar perturbaciones externas moderadas y ser tolerante a las discrepancias producidas por dinámicas no modeladas (fricciones no lineales y holguras mecánicas).
- **RNF3. Restricciones de seguridad (Gestión de fallos):**
 - *Seguridad mecánica:* El software debe implementar una parada de emergencia si las lecturas de posición indican que el carro excede los límites físicos del riel. En caso de fallo en el software de parada se debe disponer de finales de carrera para cortar la alimentación del actuador.
 - *Seguridad eléctrica:* Se debe garantizar el aislamiento galvánico entre la etapa de control (3.3V) y la etapa de potencia (24V).
- **RNF4. Condiciones de implementación (Abstracción HAL):** El código desarrollado debe hacer uso de la capa de abstracción de hardware (HAL) ofrecida por el fabricante del microcontrolador, asegurando la portabilidad, estandarización y eficiencia del firmware.

2.3 Diseño del sistema

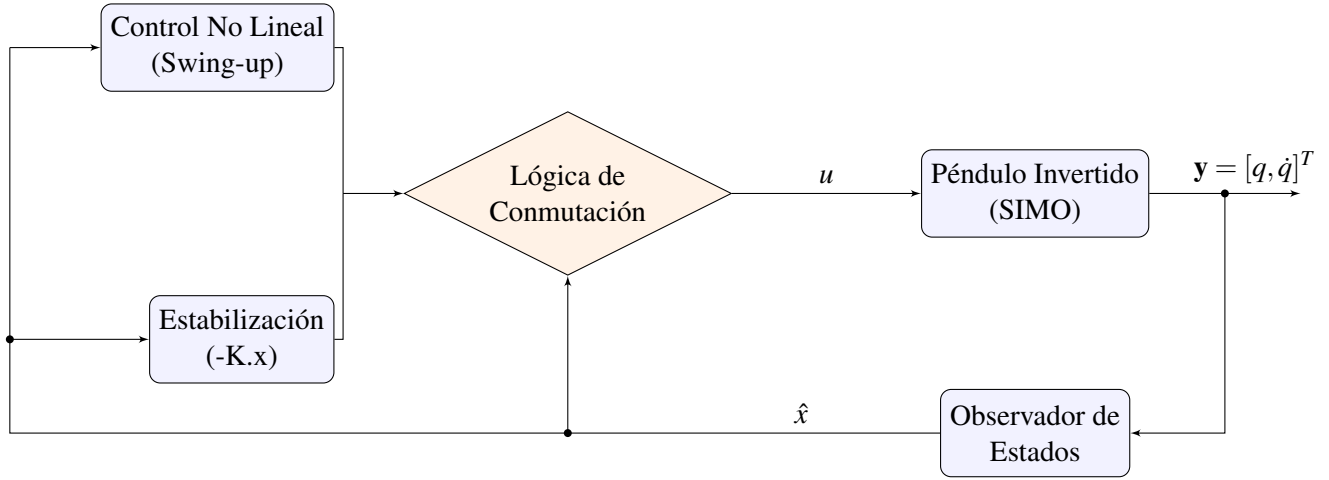


Figura 2.1: Arquitectura de control

La arquitectura de control propuesta en la Figura 2.1 se aplica sobre un sistema subactuado tipo SIMO (Single Input Multiple Output). Para la etapa de realimentación, se implementa un observador del estado por inversión. Mediante este enfoque, los estados del sistema se obtienen exclusivamente a partir de las mediciones de salida, dado que, en el caso concreto de sistemas robóticos, el estado coincide con las salidas cinemáticas $(q, \dot{q})$.

2.3.1 Introducción al modelado dinámico

A diferencia de la cinemática, que estudia el movimiento de un mecanismo sin considerar las fuerzas y pares que lo producen, el modelado dinámico describe explícitamente la relación entre la fuerza aplicada y el movimiento resultante [3]. Disponer de las ecuaciones de movimiento exactas es un requisito estricto y fundamental tanto para la simulación del sistema como para el diseño de controladores basados en modelo.

Para determinar la dinámica del péndulo invertido sobre carro, se recurre a las ecuaciones de Euler-Lagrange. Este enfoque energético se basa en la obtención del Lagrangiano del sistema (L), definido como la diferencia entre la energía cinética (E_c) y la energía potencial (E_p):

$$L = E_c - E_p \quad (2.1)$$

Las ecuaciones diferenciales no lineales resultantes pueden reescribirse y compactarse de forma elegante en la formulación matricial clásica para sistemas robóticos [4]:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = Bu \quad (2.2)$$

Para el sistema en concreto, las dimensiones de cada matriz son:

- $M(q) \in \mathbb{R}^{2 \times 2}$ matriz de inercia.
- $C(q, \dot{q}) \in \mathbb{R}^{2 \times 2}$ matriz de fuerzas centrípetas y de Coriolis. Incluye también el rozamiento viscoso.
- $G(q) \in \mathbb{R}^{2 \times 1}$ vector de fuerzas gravitacionales.
- $B \in \mathbb{R}^{2 \times 1}$ vector de entrada, mapea la acción de control u .

Este enfoque matricial no solo facilita el análisis teórico, sino que permite realizar correcciones en la simulación del sistema muy fácilmente, ya que los parámetros que lo caracterizan (masas, longitudes, inercias, etc.) se encapsulan dentro de las propias matrices.

2.3.2 Modelo dinámico del sistema

El primer paso para el modelado dinámico consiste en definir un sistema de referencia en el plano bidimensional (x,y) . Esto permite localizar la posición del centro de masas (CM) del péndulo en función de las coordenadas generalizadas del sistema: la posición lineal del carro (x) y la posición angular del péndulo (θ) .

Considerando l como la distancia desde el eje de rotación hasta el centro de masas del péndulo, sus coordenadas son:

$$\begin{cases} x_{cm} = x + l \sin(\theta) \\ y_{cm} = -l \cos(\theta) \end{cases} \quad (2.3)$$

Esta elección de coordenadas establece de forma implícita el marco de trabajo del controlador. Se define el péndulo colgando hacia abajo como el origen angular ($\theta = 0$ rad) y de acuerdo con la geometría de la Ecuación 2.3, se asume una convención de signos donde el incremento de θ es en sentido antihorario.

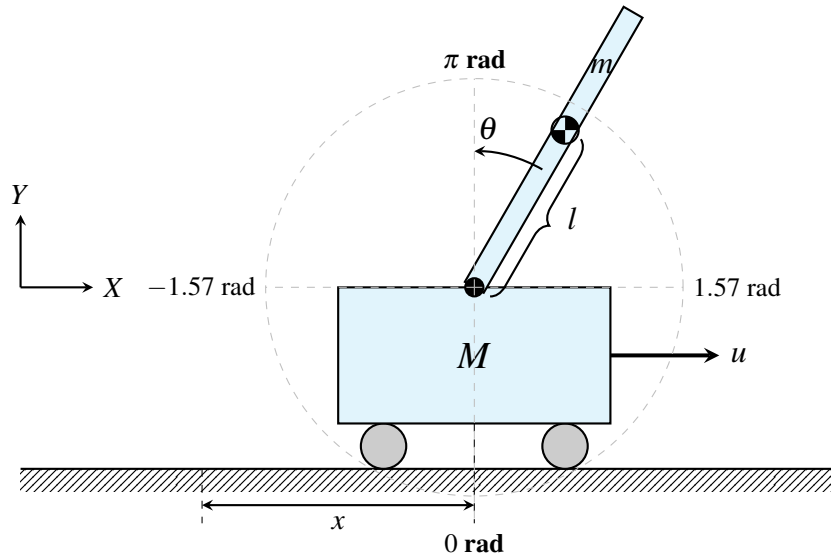


Figura 2.2: Esquema cinemático, definición del sistema de coordenadas

Una vez establecida la convención de signos del sistema y tener una visión esquemática ofrecida en la Figura 2.2, es momento de desarrollar la ecuación 2.1 hasta obtener la forma matricial compacta de la Ecuación 2.2. La energía potencial y cinética del sistema en concreto se puede representar como [5]:

$$\begin{cases} E_c = \frac{1}{2}(M+m)\dot{x}^2 + m\dot{x}\dot{\theta}l \cos(\theta) + \frac{1}{2}ml^2\dot{\theta}^2 \\ E_p = -mgl \cos(\theta) \end{cases} \quad (2.4)$$

A partir de formular la energía cinética y potencial, se desarrolla el Lagrangiano (Anexo A) y se calculan sus derivadas parciales hasta obtener las siguientes ecuaciones de movimiento [5]:

$$\begin{cases} (M+m)\ddot{x} + ml\ddot{\theta} \cos(\theta) - ml\dot{\theta}^2 \sin(\theta) = u \\ ml\ddot{x} \cos(\theta) + ml^2\ddot{\theta} + mgl \sin(\theta) = 0 \end{cases} \quad (2.5)$$

Antes de compactar las ecuaciones en la forma matricial de la Ecuación 2.2 es importante añadir los términos de rozamiento viscoso para el riel y el péndulo.

$$\begin{cases} (M+m)\ddot{x} + ml\ddot{\theta}\cos(\theta) + c_x\dot{x} - ml\dot{\theta}^2\sin(\theta) = u \\ ml\ddot{x}\cos(\theta) + ml^2\ddot{\theta} + c_\theta\dot{\theta} + mgl\sin(\theta) = 0 \end{cases} \quad (2.6)$$

Finalmente se compacta la expresión en forma matricial como indica la Ecuación 2.2. La caracterización del sistema que posteriormente se implementará en simulación es:

$$\begin{bmatrix} M+m & ml\cos\theta \\ ml\cos\theta & I+ml^2 \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} + \begin{bmatrix} c_x & -ml\dot{\theta}\sin\theta \\ 0 & c_\theta \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ mgl\sin\theta \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} u \quad (2.7)$$

Si se analiza la Ecuación 2.7 se deducen tres características clave del sistema:

- **Sistema subactuado:** La acción de control u solo permite empujar el carro de forma horizontal, no hay control directo en el posicionado del péndulo.
- **Acoplamiento de términos:** El movimiento del carro afecta a la posición del péndulo y a su vez el movimiento del péndulo actúa como una perturbación en el posicionado del carro.
- **Sistema inestable:** Si se analiza el sistema en $\theta = \pi \text{ rad}$ presenta polos con parte real positiva debido al término $mgl\sin\theta$. Por tanto, se requiere aplicar acciones de control en bucle cerrado para poder permanecer en esta posición.

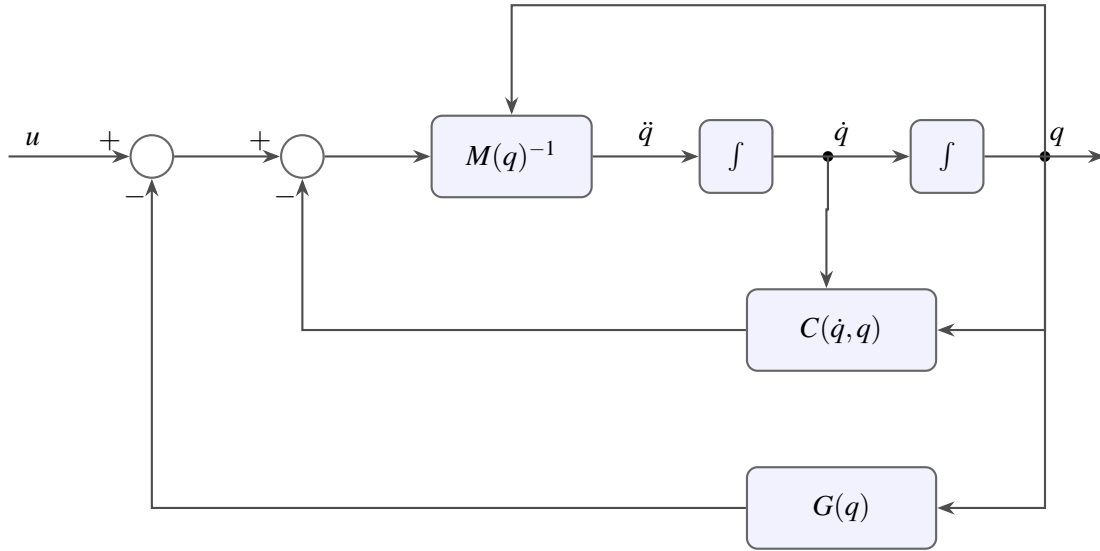


Figura 2.3: Diagrama de bloques del modelado

Finalmente, en la Figura 2.3 se muestra la representación del modelado dinámico del sistema con diagramas de bloques. Es una forma interesante de representación, ya que la traslación a *Simulink* es directa, sirviendo como preámbulo para la implementación de la simulación en este entorno.

2.3.3 Estrategia de control Swing-up

La primera ley de control se centra en el levantamiento del péndulo desde su posición estable ($\theta = 0 \text{ rad}$) hasta su posición inestable y punto en el que se conmuta al control de posición ($\theta = \pi \text{ rad}$).

La idea intuitiva tras las matemáticas es simple, consiste en balancear el péndulo inyectando energía a favor del movimiento. El objetivo es llegar al punto de estabilización con pequeñas acciones de control que producen oscilaciones de amplitud creciente en el sistema.

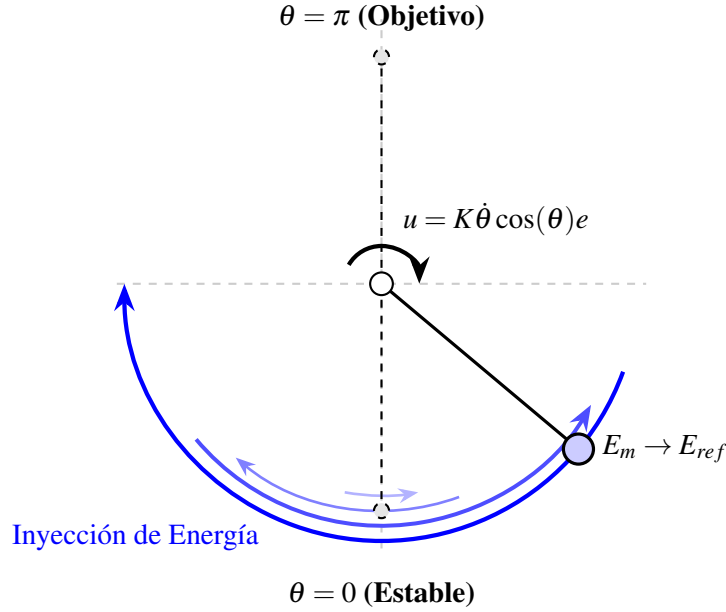


Figura 2.4: Esquema del control por balanceo (swing-up) basado en energía.

Para formalizar la ley de control esquematizada en la Figura 2.4, se define la energía del péndulo libre que viene dada por la suma de la energía potencial y la cinética [6]:

$$E_m = E_c + E_p = \frac{1}{2}J\dot{\theta}^2 - mgl \cos(\theta) \quad (2.8)$$

En el caso de la Ecuación 2.8 se define la máxima energía potencial cuando $\theta = \pi \text{ rad}$ de forma que el sistema de referencia del control coincide con el del modelado. Una vez se formaliza el modelo energético, se determina la referencia energética que el control por balanceo debe conseguir [6]:

$$E_{ref} = mgl \quad (2.9)$$

Por otro lado, el error de seguimiento para calcular la acción de control u viene dado por la diferencia entre la energía actual del péndulo (E_m , Ecuación 2.8) y la energía de referencia (E_{ref} , Ecuación 2.9) [6]:

$$e = E_m - E_{ref} \quad (2.10)$$

Finalmente se define la ley de control que permite balancear el péndulo simple hasta su posición inestable, donde la acción de control u es el par aplicado en el eje de rotación [6]:

$$u = K\dot{\theta} \cos(\theta)e \quad (2.11)$$

En función del valor de la ganancia K se conseguirá el objetivo en diferente número de oscilaciones. Valores pequeños de la ganancia requieren de más iteraciones para llegar hasta el punto de estabilización.

Sin embargo, la ley de control definida en la Ecuación 2.11 no es directamente transferible al péndulo invertido sobre carro. Los motivos son los siguientes:

- **Acción de control:** En la ley de control propuesta en 2.11 el actuador se encuentra en el eje del péndulo. El sistema definido en 2.7 es subactuado y solo se tiene control sobre la fuerza de empuje del carro. Para aplicar la estrategia de energía, es necesario convertir la acción de control virtual (el par necesario en el eje) a una aceleración real del carro a través del acoplamiento dinámico del sistema.
- **Límites físicos del riel:** Se hace indispensable, por tanto, un control multiobjetivo que regule la energía del péndulo garantizando simultáneamente la estabilidad de la posición del carro dentro del espacio de trabajo del riel.

Para abordar las limitaciones mencionadas, se recurre a la técnica de **Linearización por Realimentación Parcial (PFL)**[7]. Permite transformar linealizar la dinámica de un subconjunto del sistema mediante un cambio de variables y una ley de control no lineal, siempre que exista un acoplamiento inercial suficiente entre las coordenadas actuadas y las pasivas [7].

En este trabajo se aplica la variante conocida como **linealización colocalizada** (*Collocated Linearization*), la cual consiste en linealizar el grado de libertad activo del sistema, es decir, aquel donde se ubica físicamente la acción de control [7]. El objetivo es diseñar una ley de control $u(N)$ que cancele las no linealidades de la dinámica del carro para que este se comporte, desde el punto de vista del controlador de alto nivel, como un doble integrador respecto a una aceleración deseada $\ddot{x}_{des}(\frac{m}{s^2})$.

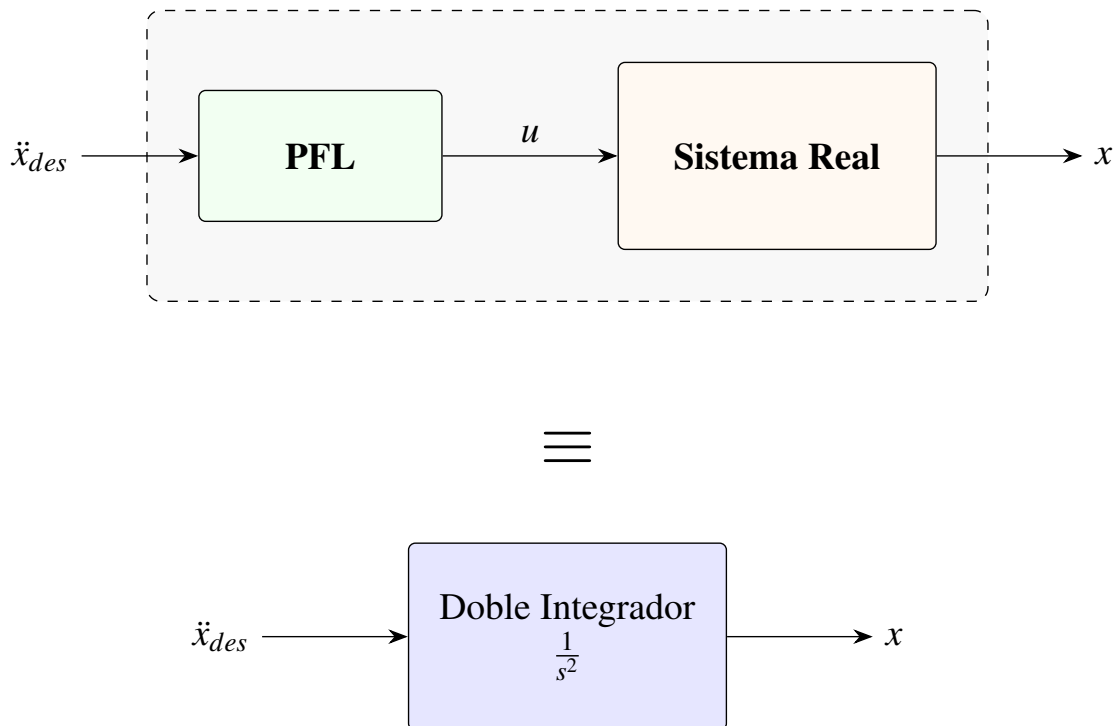


Figura 2.5: Concepto de equivalencia de la Linealización Colocalizada.

Partimos de las ecuaciones de movimiento del sistema que incluyen los términos de fricción del carro (c_x) y del péndulo (c_θ):

$$\begin{cases} (M+m)\ddot{x} + ml\ddot{\theta}\cos(\theta) + c_x\dot{x} - ml\dot{\theta}^2\sin(\theta) = u \\ ml\ddot{x}\cos(\theta) + J\ddot{\theta} + c_\theta\dot{\theta} + mgl\sin(\theta) = 0 \end{cases} \quad (2.12)$$

Donde $J = I + ml^2$ es el momento de inercia del péndulo respecto al eje de rotación. Para eliminar $\ddot{\theta}$ de la ecuación de control, se despeja dicha variable de la dinámica del péndulo [5]:

$$\ddot{\theta} = \frac{1}{J} (-mgl\sin\theta - ml\cos\theta\ddot{x} - c_\theta\dot{\theta}) \quad (2.13)$$

Al imponer la condición de linealización $\ddot{x} = \ddot{x}_{des}$ y sustituir la expresión 2.13 en la ecuación del carro, se obtiene la fuerza u necesaria para anular la dinámica no lineal del subsistema actuado [5]:

$$u = (M+m)\ddot{x}_{des} + ml\cos\theta \left[\frac{1}{J} (-mgl\sin\theta - ml\cos\theta\ddot{x}_{des} - c_\theta\dot{\theta}) \right] - ml\dot{\theta}^2\sin\theta + c_x\dot{x} \quad (2.14)$$

Como se observa, la fuerza física aplicada (u) compensa dinámicamente la masa total y los efectos inerciales del péndulo. Esta arquitectura permite definir la aceleración deseada $\ddot{x}_{des}$ como el punto de encuentro donde convergen los dos objetivos de control del sistema [5]:

$$\ddot{x}_{des} = \underbrace{K_e\dot{\theta}\cos\theta(E - E_{ref})}_{\text{Control de Energía (Swing-up)}} - \underbrace{(K_p x + K_d \dot{x})}_{\text{Control de Posición (PD)}} \quad (2.15)$$

La ley de control resultante en la ecuación 2.15 simboliza un compromiso entre la inyección de energía necesaria para el balanceo y la regulación de la posición del carro sobre el riel [5]. Se debe ajustar el peso de cada ganancia para obtener un equilibrio óptimo entre ambos controles.

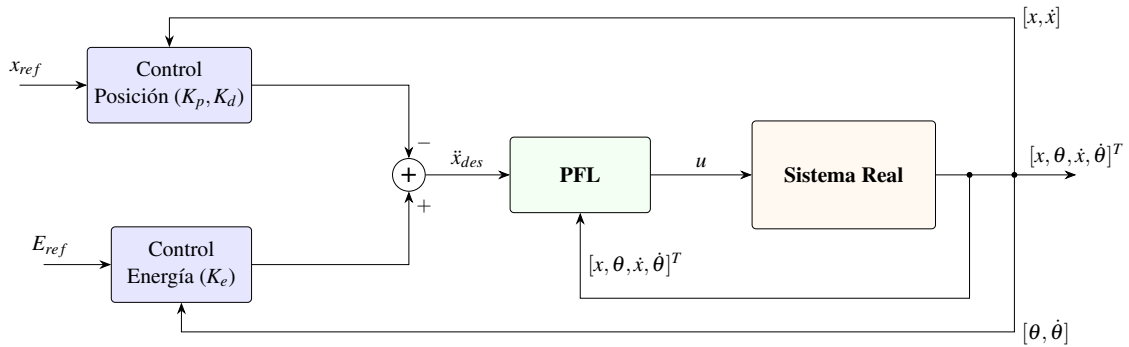


Figura 2.6: Diagrama de bloques de la estrategia de Swing-up con linealización parcial (PFL)

2.3.4 Modelado en el espacio de estados

Una vez que la estrategia de *swing-up* ha llevado al péndulo a su posición vertical inestable ($\theta \approx \pi \text{ rad}$), se requiere mantener el péndulo equilibrado en su punto superior mientras se regula la posición del carro en un punto de referencia (típicamente el centro del riel, $x = 0$). Para lograr el cometido se opta por aplicar un control por realimentación del estado. Un control de este tipo, requiere una representación en el espacio de estados del modelo dinámico linealizado en el punto de equilibrio. Todos los sistemas bajo este marco se definen como [8]:

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases} \quad (2.16)$$

Donde el vector de estados $\mathbf{x}(t) = [x, \theta, \dot{x}, \dot{\theta}]^T \equiv [q, \dot{q}]^T$ define completamente la condición del sistema en cada instante. Los componentes de la representación se describen a continuación [8]:

- **Matriz de estado** ($A \in \mathbb{R}^{4 \times 4}$): Caracteriza la dinámica interna del sistema y cómo evolucionan los estados (posiciones y velocidades) entre sí cuando no hay acción de control. Sus autovalores son los polos del sistema e indican si es estable.
- **Matriz de entrada** ($B \in \mathbb{R}^{4 \times 1}$): Define cómo la fuerza aplicada al carro (u) afecta a la aceleración de cada uno de los estados.
- **Matriz de salida** ($C \in \mathbb{R}^{4 \times 4}$): Relaciona los estados internos con las variables medidas. Dado que en este proyecto se dispone de sensores para todas las variables, C se simplifica a la matriz identidad I .
- **Matriz de transmisión directa** (D): Representa el acoplamiento directo entre la entrada y la salida. Para este caso D es cero, ya que la acción de control actúa sobre la dinámica del sistema primero.

Para sistematizar la transición entre el modelo físico no lineal y el diseño del controlador de estabilidad, se parte de la ecuación 2.2. Al aplicar una linealización por Jacobiano alrededor del punto de equilibrio inestable $x = [0, \pi, 0, 0]^T$, las matrices de la dinámica se transforman en matrices de coeficientes constantes evaluadas en dicho punto [8]:

- **Inercia:** $\bar{M} = M(\bar{q})$
- **Amortiguamiento:** $\bar{C} = C(\bar{q}, 0)$
- **Rigidez (Gravedad):** $\bar{G} = \left. \frac{\partial G(q)}{\partial q} \right|_{q=\bar{q}}$
- **Entrada:** $\bar{B} = B(\bar{q})$

Para el caso en particular del péndulo invertido sobre carro, el sistema linealizado en dicho punto es el siguiente:

$$\underbrace{\begin{bmatrix} M+m & -ml \\ -ml & I+ml^2 \end{bmatrix}}_{\bar{M}} \underbrace{\begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix}} + \underbrace{\begin{bmatrix} c_x & 0 \\ 0 & c_\theta \end{bmatrix}}_{\bar{C}} \underbrace{\begin{bmatrix} \dot{x} \\ \dot{\theta} \end{bmatrix}} + \underbrace{\begin{bmatrix} 0 \\ -mgl \end{bmatrix}}_{\bar{G}} = \underbrace{\begin{bmatrix} 1 \\ 0 \end{bmatrix}}_{\bar{B}} u \quad (2.17)$$

El sistema descrito en la ecuación 2.17 es bajo el que se diseñará el control de posición. Contra más lejos esté el péndulo de su vertical menos efectivo será el control, ya que las diferencias del

sistema real respecto del lineal empiezan a acentuarse, este es uno de los motivos por el que se hace necesaria la conmutación entre control de posición y swing-up.

Definiendo el vector de estados como $\mathbf{x} = [q, \dot{q}]^T$, el sistema puede reescribirse en la representación compacta de espacio de estados mediante la siguiente expresión [8]:

$$\begin{bmatrix} \dot{q} \\ \ddot{q} \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{0} & I \\ -\bar{M}^{-1}\bar{G} & -\bar{M}^{-1}\bar{C} \end{bmatrix}}_A \begin{bmatrix} q \\ \dot{q} \end{bmatrix} + \underbrace{\begin{bmatrix} \mathbf{0} \\ \bar{M}^{-1}\bar{B} \end{bmatrix}}_B u \quad (2.18)$$

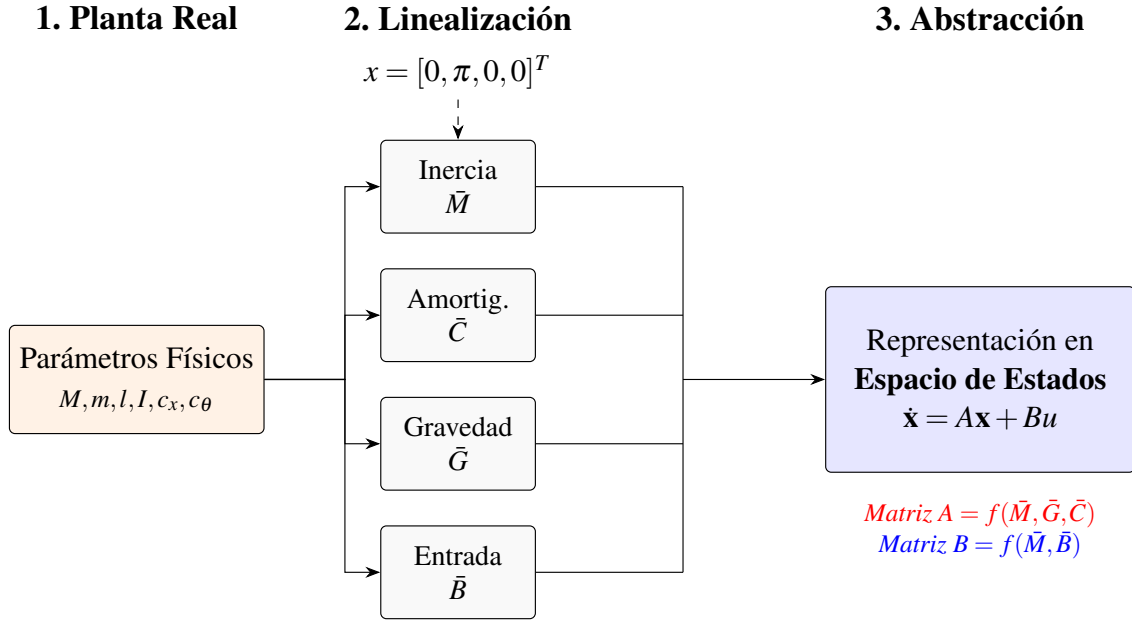


Figura 2.7: Flujo de trabajo para el modelado

Las principales ventajas de aplicar esta técnica para la obtención de la representación del sistema en el espacio de estados son:

- **Abstracción y generalización del modelado:** La técnica es escalable a cualquier sistema que cumpla la dinámica definida en la ecuación 2.2. Permite centrarse en el diseño del controlador en lugar de obtener de forma manual las matrices A y B para cada sistema concreto.
- **Eficiencia en la implementación:** La transición a un marco matricial facilita enormemente la implementación en lenguajes de programación de alto nivel, como *MATLAB* o *Python* (mediante la librería *NumPy*). El programador simplemente define las matrices ($\bar{M}, \bar{C}, \bar{G}$ y $\bar{B}$) luego el propio lenguaje obtiene en la salida la representación en espacio de estados (A y B).

2.3.5 Control por realimentación del estado

Tras definir en la anterior sección el *sistema lineal invariante (LTI)* que define la dinámica del sistema en el punto de equilibrio $x = [0, \pi, 0, 0]^T$, se requiere diseñar una ley de control que permita estabilizar el sistema en esta región.

El **control por realimentación del estado** para este caso asume que todos los estados del sistema ($\mathbf{x}$) están disponibles para su medición. Bajo esta premisa, la acción de control $u(t)$ se calcula como una combinación lineal de dichos estados multiplicados por un vector de ganancias K , sumado opcionalmente a un término de referencia $r(t)$ escalado por una ganancia K_r [9]:

$$u(t) = K_r r(t) - K \mathbf{x}(t) \quad (2.19)$$

Desglosando el vector de ganancias $K = [K_1, K_2, K_3, K_4]$, la fuerza aplicada al carro se define de forma extendida como:

$$u = K_r r - (K_1 x + K_2 \theta + K_3 \dot{x} + K_4 \dot{\theta}) \quad (2.20)$$

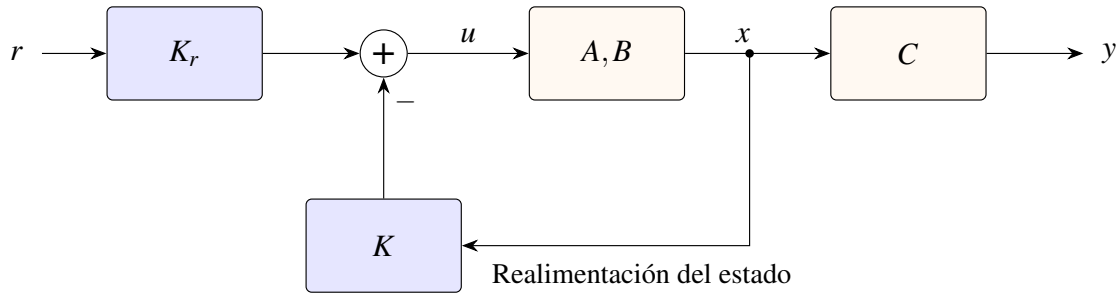


Figura 2.8: Diagrama de bloques de la ley de control por realimentación del estado

La figura 2.8 muestra como el controlador obtiene la acción de control a aplicar en cada instante de tiempo en función de los estados y la referencia. Por otro lado, en comparación con la ecuación 2.16, el sistema en bucle cerrado representado en espacio de estados es [9]:

$$\begin{cases} \dot{x}(t) = Ax(t) + B(K_r \cdot r(t) - K \cdot x(t)) = (A - BK)x(t) + BK_r r(t) \\ y(t) = Cx(t) + Du(t) \end{cases} \quad (2.21)$$

Bajo esta configuración, la dinámica del sistema en bucle cerrado queda gobernada por la expresión matricial $(A - BK)$. Por tanto, el diseño del controlador se reduce a determinar los valores del vector de ganancias $K = [k_1, k_2, k_3, k_4]$ que garanticen que todos los autovalores resultantes posean parte real negativa, dotando así al sistema de una respuesta temporal específica. Para abordar este problema, se consideran habitualmente dos metodologías:

- **Asignación de polos:** Consiste en definir arbitrariamente la ubicación de los autovalores de la matriz $(A - BK)$ en el plano complejo. Se calculan las ganancias de K para que el sistema responda con una dinámica concreta (amortiguamiento y tiempo de establecimiento) [9].

- **Regulador Cuadrático Lineal (LQR):** Esta metodología aborda el diseño como un problema de optimización. En lugar de elegir la ubicación de los polos, el diseñador define qué es más costoso: que el sistema se aleje del equilibrio o que el actuador trabaje en exceso [9]. El algoritmo calcula de forma automática la matriz de ganancias K que minimiza el siguiente índice de coste cuadrático:

$$J = \int_0^{\infty} (\mathbf{x}^T Q \mathbf{x} + u^T R u) dt \quad (2.22)$$

El comportamiento del controlador se sintoniza a través de dos matrices de ponderación:

- **Matriz Q (4×4):** Penaliza la desviación de los estados. Valores elevados en la diagonal de Q indican que se prioriza la precisión y la rapidez en la estabilización de los estados $(x, \theta, \dot{x}, \dot{\theta})$.
- **Matriz R (1×1):** Penaliza el esfuerzo de control $u(t)$. Un valor de R elevado suaviza la respuesta del actuador.

Es importante notar que la respuesta del sistema no depende de los valores absolutos de las matrices, sino del peso relativo entre ellas (Q frente a R).

2.3.6 Lógica de conmutación

Finalmente, la lógica de conmutación atiende a un umbral angular. Este bloque de decisión monitoriza de forma continua la posición angular estimada del péndulo ($\hat{\theta}$). Si el péndulo ingresa dentro de una vecindad acotada en torno a la vertical absoluta (*el punto de linealización del sistema*), la lógica conmuta al control por realimentación del estado. En caso contrario, el sistema mantiene activo el controlador de swing-up para elevar el mecanismo. Este comportamiento híbrido se formaliza en el siguiente pseudocódigo:

Algoritmo 1 Lógica de Conmutación

Require: Vector de estados estimado $\hat{x} = [\hat{x}_c, \hat{x}_c, \hat{\theta}, \hat{\theta}]^T$, umbral angular θ_{th}

Ensure: Señal de control u a aplicar a la planta

```

1: loop
2:    $\hat{\theta} \leftarrow \hat{x}(3)$                                 ▷ Obtener posición angular actual
3:   if  $|\hat{\theta}| \leq \theta_{th}$  then
4:      $u \leftarrow -K\hat{x}$                                 ▷ Control de posición
5:   else
6:      $u \leftarrow f_{swing}(\hat{x})$                         ▷ Control de energía
7:   end if
8:
9: end loop

```

2.4 Arquitectura del sistema

Para definir la arquitectura del sistema se ha hecho una división entre la parte de simulación y la implementación en un sistema real.

La arquitectura del sistema de simulación permite obtener una vista general del flujo de trabajo para el modelado y la validación antes de la implementación real. Por otro lado, la arquitectura del sistema real se centra en los periféricos involucrados en el control así como el proceso de discretización que se debe aplicar para controlar un sistema real utilizando un microcontrolador.

2.4.1 Arquitectura de la simulación

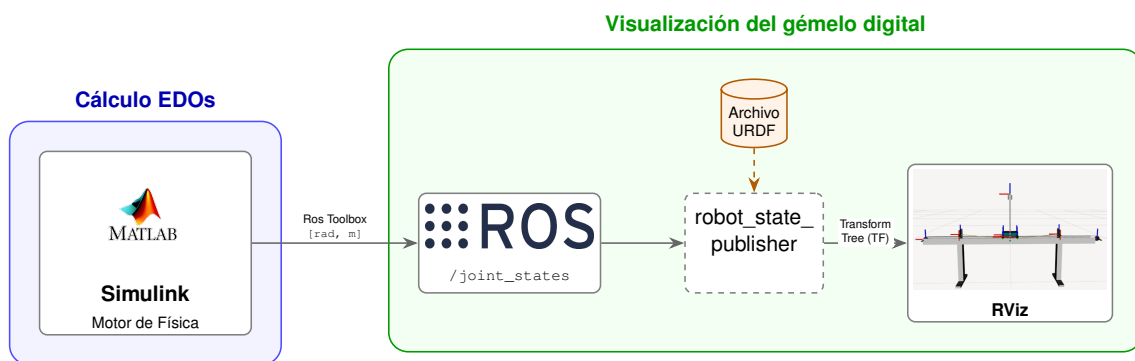


Figura 2.9: Arquitectura de la simulación

La Figura 2.9 detalla la arquitectura de software diseñada para la simulación del sistema. A continuación se desglosa cada elemento del esquema para conocer su función y como se conecta con el resto de la arquitectura:

- **Cálculo de EDOs (Motor de Física):** Implementado en el entorno MATLAB/Simulink, este bloque se encarga de resolver las Ecuaciones Diferenciales Ordinarias (EDO) que describen la dinámica del sistema. Simulink actúa como el motor de física, calculando en cada paso de tiempo los estados del robot (posiciones en radianes y metros). Estos datos se transmiten al ecosistema ROS mediante ROS Toolbox, publicando la información en el tópico `/joint_states`.
- **Visualización, gemelo digital:** Este bloque recibe la información cinemática para representar el comportamiento del sistema en tiempo real. Está totalmente integrado en el entorno de ROS y se compone de varias piezas:
 - **Archivo URDF (Unified Robot Description Format):** Es un archivo XML que describe como es el robot o mecanismo que se quiere simular. Se especifican las visuales, las restricciones cinemáticas y los marcos de referencia de las articulaciones.
 - **Robot State Publisher:** Es el nodo de ROS que permite representar el movimiento del robot. Se suscribe al tópico `/joint_states` y actualiza el árbol de transformadas de nuestro archivo URDF.
 - **RViz:** La última pieza de la arquitectura software, es la interfaz que permite visualizar de forma gráfica e intuitiva todo el flujo de datos como un modelo 3D en movimiento.

2.4.2 Arquitectura del sistema real

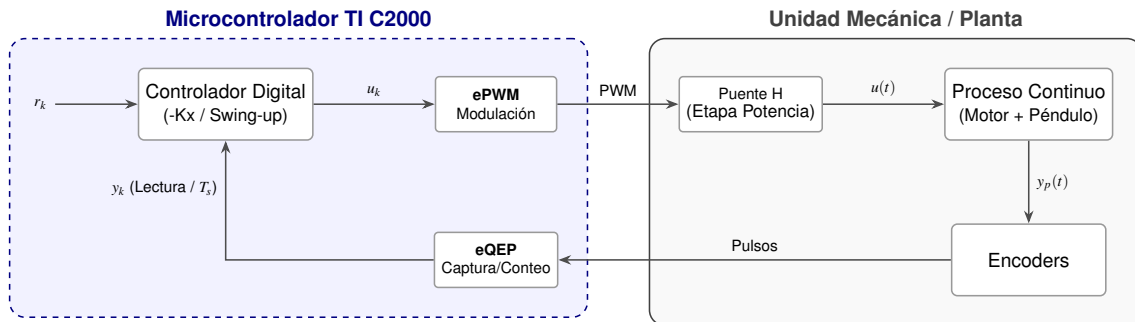


Figura 2.10: Arquitectura del sistema real

La Figura 2.10 presenta de forma esquemática la implementación del bucle de control en el microcontrolador.

Esta estructura permite observar la interacción entre el entorno digital (discreto) y el proceso físico (continuo). El núcleo del sistema es el **Controlador Digital**. Este procesador se encarga de ejecutar el algoritmo de control que procesa la señal de error entre la referencia (r_k) y la realimentación (y_k).

El concepto fundamental es el **periodo de muestreo** (T). Este parámetro determina la periodicidad de la rutina de control, permitiendo sincronizar las etapas de medida, cálculo y actuación sobre la planta de manera precisa. Durante el desarrollo del proyecto se determinará el periodo de muestreo atendiendo a factores como la dinámica del sistema en bucle cerrado o la frecuencia de la señal PWM.

Finalmente la interacción entre el mundo digital y el mundo físico es posible mediante el uso de **periféricos** que permiten medir el estado del sistema o modificarlo. Diferenciamos en dos clases de periféricos:

- **Actuadores:** La señal de control digital calculada (u_k) se traduce en señales lógicas de dirección y una señal modulada por ancho de pulso (**PWM**) generada por el periférico **ePWM**. Tanto el estado de las señales digitales como el ciclo de trabajo (*duty cycle*) del PWM se mantienen constantes durante el intervalo de muestreo (T_s). Estas señales excitan un puente en H (driver), el cual conmuta el voltaje de alimentación para inyectar una corriente en el motor. De este modo, se transforma la decisión del algoritmo en el par mecánico necesario para accionar el proceso continuo.
- **Sensores:** La respuesta física del sistema ($y_p(t)$) es captada por encoders ópticos incrementales. A diferencia de un sensor analógico convencional, estos dispositivos generan una secuencia de pulsos basada en eventos de movimiento (detección de flancos). El periférico **eQEP** del microcontrolador actúa como una unidad de conteo especializada que registra estos eventos de forma asíncrona. No obstante, para el bucle de control, la señal se digitaliza de forma efectiva (y_k) al leer el estado de dichos contadores en intervalos de tiempo estrictamente periódicos (T_s). Este muestreo periódico es fundamental para el cálculo de variables derivadas, como la velocidad, mediante la diferencia de posición en cada intervalo.

2.4.3 Especificación de componentes de hardware

Para la implementación física del sistema de control diseñado, se han seleccionado componentes que garanticen un compromiso óptimo entre precisión de lectura, capacidad de procesamiento y tiempo de respuesta. En la Tabla 2.1 se resumen los elementos principales del prototipo.

Cuadro 2.1: Resumen de componentes del sistema de hardware.

Componente	Modelo	Función Principal
Microcontrolador	TMS320F280049C	Procesamiento de señales y ejecución del control
Driver de motor	AQMH2407ND	Interfaz de potencia (Puente H con aislamiento)
Encoder Motor	HEDS-5140-I (2048 PPR)	Medición de posición lineal (x)
Encoder Péndulo	HEDS-5140-A06 (2000 PPR)	Medición de ángulo vertical (θ)
Actuador	Crouzet 82 760 0 (24V)	Generación de fuerza sobre el carro (u)

Unidad de Procesamiento (Microcontrolador)

Para la implementación del algoritmo de control se ha seleccionado el **TMS320F280049C**, un microcontrolador de tiempo real de la familia **C2000** de **Texas Instruments**. Este dispositivo está específicamente diseñado para aplicaciones de control, destacando su núcleo **C28x** a **100 MHz** que integra una unidad de punto flotante (**FPU**) y una unidad de aceleración trigonométrica (**TMU**) [10]. Ambos aceleradores son fundamentales para procesar la aritmética del controlador y las funciones trigonométricas del modelo.

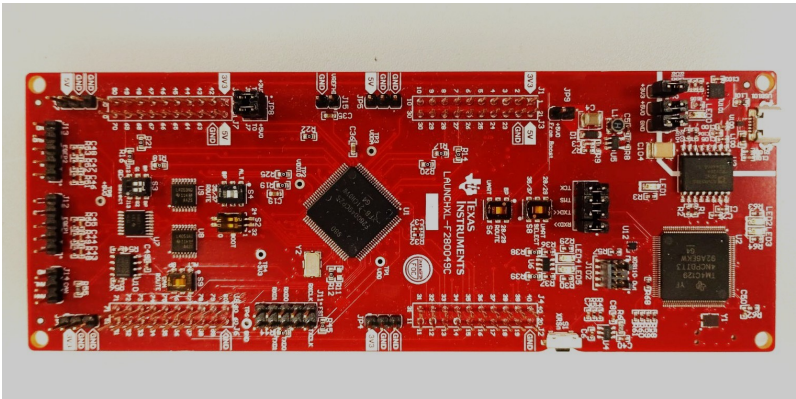


Figura 2.11: TMS320F280049C en formato *Launchpad*

Asimismo, el chip dispone de periféricos de control avanzado que descargan de tareas críticas a la CPU, los más destacados en el proyecto son:

- **eQEP (Enhanced Quadrature Encoder Pulse):** Dos módulos dedicados para la decodificación de señales en cuadratura de los encoders, permitiendo una lectura de posición y ángulo sin consumo de ciclos de reloj [10].
- **ePWM (Enhanced Pulse Width Modulator):** El periférico permite una resolución de **16 bits** en la generación del ciclo de trabajo (*duty cycle*) y hasta **32 bits** si se utiliza *High Resolution PWM*, garantizando un control fino sobre el motor DC [10].

El desarrollo software se apoya en el ecosistema oficial de Texas Instruments. Como entorno de desarrollo integrado (IDE) se emplea **Code Composer Studio (CCS)** [11], permite una depuración avanzada y optimización específica para la arquitectura C28x. Además, se utiliza la **SDK C2000Ware**, que proporciona una capa de abstracción de hardware (HAL) facilitando la programación y la interpretabilidad del código [12].

Etapas de Potencia y Actuación

La acción de control $u(t)$ se traduce en una tensión aplicada al motor mediante el driver de motores de corriente continua **AQMH2407ND**. Implementa un **Puente H** de doble canal basado en tecnología **MOSFET** [13], lo que le otorga una eficiencia energética superior y una disipación térmica significativamente menor en comparación con drivers basados en transistores **BJT**, como el **L298N** [14].

El dispositivo es capaz de suministrar una corriente continua de hasta **7 A** por canal (con picos de hasta 50 A) [13]. El control se gestiona mediante señales PWM provenientes del microcontrolador, permitiendo la regulación de la tensión media aplicada al motor.

Un aspecto fundamental en el diseño de este driver es la integración de **optoacopladores EL357N-G** [15], los cuales proporcionan un **aislamiento galvánico** entre la etapa de control y la de potencia. Esta característica es esencial en la implementación de sistemas de control, ya que ofrece:

- **Mitigación de ruido:** Evita que el ruido electromagnético inducido por la conmutación de las bobinas del motor afecte a las señales de medición.
- **Protección del hardware:** Asegura la integridad del microcontrolador frente a posibles picos de tensión o corrientes de retorno (*back-EMF*) provenientes de la etapa de potencia.

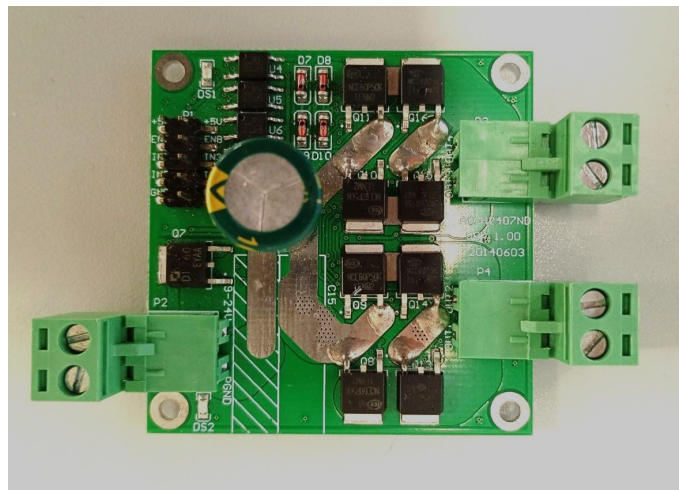


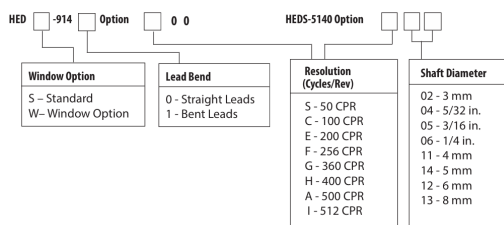
Figura 2.12: Driver AQMH2407ND

Sensores (Encoders)

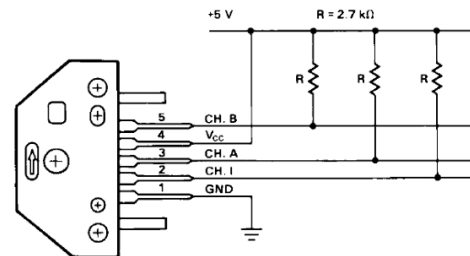
Para el caso concreto de péndulo invertido sobre carro, se requieren dos encoders (uno por grado de libertad), estos sensores son vitales para obtener el estado del sistema en cada instante de tiempo y calcular una acción de control acorde.

Concretamente, se ha utilizado un sistema de detección óptica que se compone de un módulo detector de la serie **HEDS-9140** y un disco óptico de la serie **HEDS-5140**. A continuación se describe la ubicación y resolución de los dos encoders del sistema:

- **Posición del carro (x):** Se utiliza un encoder acoplado al eje del motor de continua que proporciona una resolución de **2048 pulsos por revolución (PPR)** tras el procesado en cuadratura.
- **Ángulo del péndulo (θ):** Acoplado en el eje del péndulo, ofrece 500 ciclos por revolución (CPR), traduciendo en una resolución de **2000 PPR** en cuadratura.



(a) Especificaciones módulo de detección y disco óptico según modelo específico



(b) Descripción del módulo detector y mapeo de señales

Figura 2.13: Detalles técnicos de los componentes de detección óptica de la serie HEDS [16]

La Figura 2.13a permite identificar con precisión la configuración de cada sensor atendiendo a su resolución y dimensiones mecánicas:

- **Encoder del motor:** Para alcanzar los 2048 PPR en cuadratura, se utiliza un disco óptico de 512 CPR. Siguiendo la codificación de la figura, este corresponde al modelo **HEDS-5140-I**.
- **Encoder del péndulo:** Este eje utiliza el modelo **HEDS-5140-A06**. El código A identifica la resolución de 500 CPR (2000 PPR en cuadratura), mientras que el sufijo 06 indica un diámetro interno del disco, garantizando un ajuste mecánico con el eje sin holguras.

Por último, el módulo detector **HEDS-9140** (Figura 2.13b) es el encargado de transformar el paso de las ranuras del disco en señales TTL. La disposición de sus fotodetectores internos permite que el periférico **eQEP** del microcontrolador interprete el desfase entre los canales A y B para determinar la dirección del movimiento.

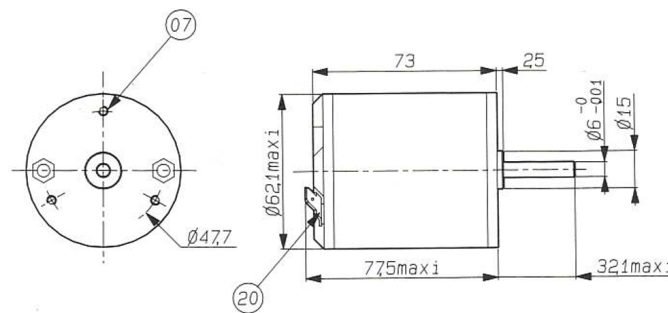
Actuador

El movimiento del carro se realiza mediante un motor de corriente continua de imanes permanentes del fabricante **Crouzet**, modelo **82 760 0**.

Cuadro 2.2: Parámetros técnicos del motor Crouzet 82 760 0 (24V) [17]

Parámetro	Valor	Unidad
Velocidad nominal	2300	rpm
Par nominal	76	mN·m
Potencia útil nominal	18	W
Corriente nominal	1,3	A
Resistencia (R)	5,5	Ω
Inductancia (L)	7,2	mH
Constante de par (K_t)	0,068	N·m/A
Inercia (J_m)	270	g·cm ²
Eficiencia (η)	57	%

Los datos del fabricante expuestos en la tabla 2.2 son vitales para caracterizar actuador y estimar la relación entre el voltaje aplicado en sus bornes y la fuerza horizontal que empuja el carro.



(a) Dimensiones del actuador [17]



(b) Motor instalado en la unidad mecánica del laboratorio

Figura 2.14: Detalles del actuador Crouzet 82 760 0: diseño técnico y montaje real.

El sistema utiliza una transmisión por correa dentada (Figura 2.14b) para convertir el movimiento rotativo del motor en el desplazamiento lineal del carro, minimizando las holguras mecánicas.

3. Desarrollo

Explicación de las actividades realizadas durante el desarrollo del proyecto. Se puede exponer en orden cronológico o por grupos de tareas. Hay que indicar los hitos alcanzados.

En esta sección solamente hay que aportar la información técnica estrictamente necesaria para comprender el trabajo realizado. Esta información debe, además, estar presentada de forma que sea inteligible por cualquiera no familiarizado con los aspectos técnicos del trabajo realizado. La información técnica puede ubicarse en los apéndices para ser consultada si se desea.

3.1 Trabajo de simulación

3.1.1 Modelado del sistema en Simulink

El punto de partida consistió en trasladar el modelo representado en la Ecuación 2.7 hacia un entorno de simulación numérica en *MATLAB/Simulink*. Para una primera aproximación, se emplearon los parámetros de masas, longitudes, inercias y rozamientos proporcionados en la documentación original del fabricante, los cuales se detallan en la Tabla 3.1. No obstante, debido a que la unidad mecánica cuenta con más de 25 años de antigüedad, posteriormente se llevó a cabo la identificación experimental de sus parámetros reales.

Cuadro 3.1: Parámetros proporcionados por el fabricante [18]

Parámetro	Símbolo	Valor	Unidades
Masa del carro	M	1.12	kg
Masa del péndulo	m	0.12	kg
Longitud al centro de masas	l	0.305	m
Inercia del péndulo	J_p	0.0136	$\text{kg} \cdot \text{m}^2$
Fricción del carro	c_x	0.05	$\text{N} \cdot \text{s}/\text{m}$
Fricción del péndulo	c_θ	0.0005	$\text{N} \cdot \text{m} \cdot \text{s}/\text{rad}$

Este enfoque metodológico responde firmemente al paradigma del *Diseño Basado en Modelos* (MBD). Disponer de un modelo matemático bien calibrado para asegurar una transición eficiente entre la teoría y la práctica. Al emular con precisión el comportamiento de la planta real, el entorno virtual permite verificar de manera exhaustiva la estabilidad y robustez de los algoritmos diseñados antes de su implementación en el hardware, mitigando por completo el riesgo de daños mecánicos o eléctricos por pérdidas de control.

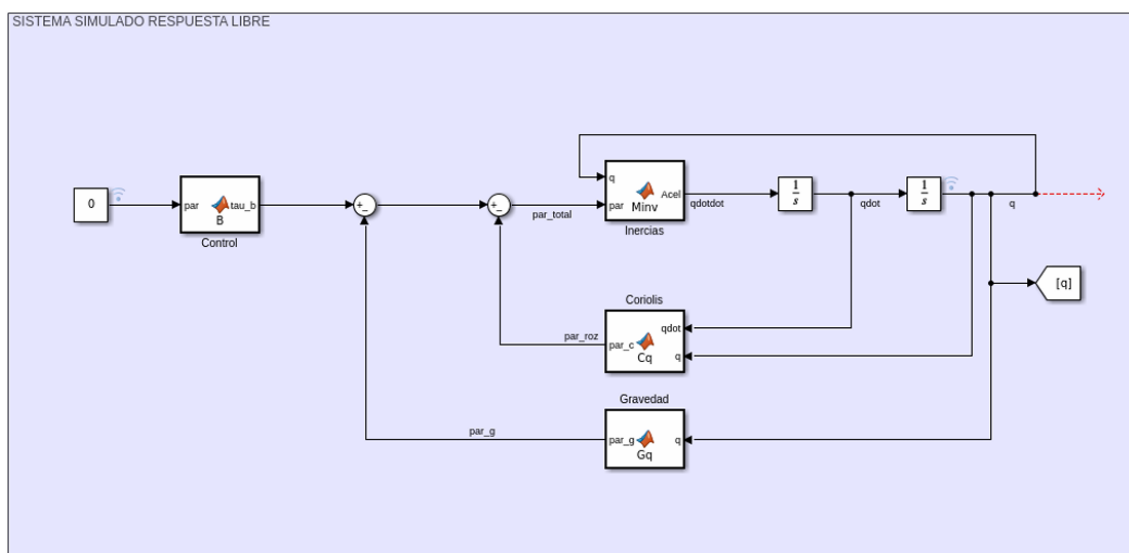


Figura 3.1: Diagrama de bloques del modelo dinámico implementado en *Simulink*

La Figura 3.1 detalla la arquitectura implementada en *Simulink*, la cual constituye la traslación directa de las ecuaciones dinámicas del sistema al entorno computacional, en el Anexo B se detalla el contenido de cada bloque. Esta estructura materializa de forma práctica el álgebra de bloques conceptual introducida previamente en la Figura 2.3, sirviendo como la plataforma sobre la cual se ejecutan los algoritmos de control por realimentación del estado y por energía.

3.1.2 Algoritmo de control en Simulink

Control de posición

Una vez modelada la dinámica del sistema de péndulo invertido sobre carro e implementada en *Simulink*, el siguiente paso fue implementar un control de posición por realimentación del estado para mantener el sistema estable con el péndulo invertido ($\theta = \pi$ rad).

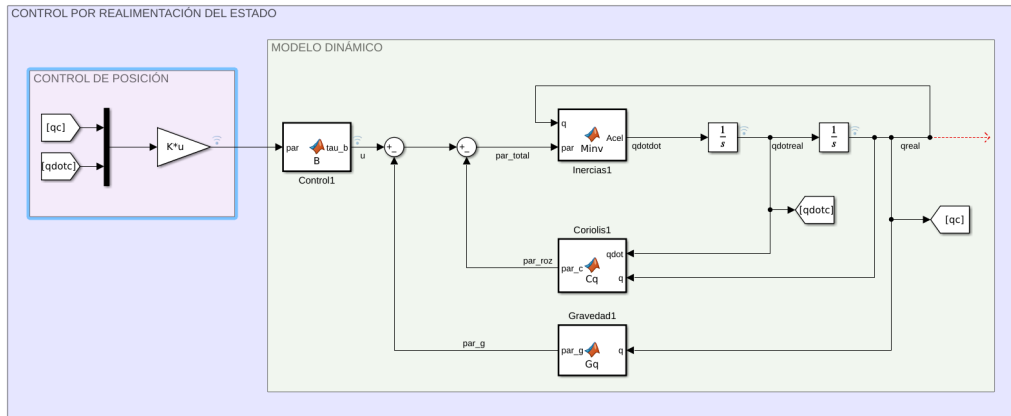


Figura 3.2: Diagrama de bloques con control de posición por realimentación del estado

Como se ilustra en la Figura 3.2, la arquitectura implementada establece una dinámica en bucle cerrado donde el bloque del controlador procesa de forma continua los vectores de estado medidos (q y $\dot{q}$). A partir de esta información, el algoritmo computa la acción de control óptima, denotada como la fuerza de empuje u . El fundamento matemático detrás de esta técnica fue abordado en el desarrollo teórico de la Sección 2.3.5.

La ley de control se reduce conceptualmente a una combinación lineal de los estados multiplicados por un vector de ganancias K , expresado formalmente como $u = K(\text{ref} - x)$.

Consecuentemente, como se aprecia en el detalle del diagrama de bloques, esta ley de control se materializa en *Simulink* mediante un único bloque de ganancia matricial (denotado en la simulación como $K*u$).

Para concluir, la obtención numérica de los vectores de ganancia K dentro del entorno de *MATLAB*, se optó por las dos metodologías clásicas de control propuestas en la Sección 2.3.5:

- **Regulador Lineal Cuadrático (LQR):** se definieron las matrices de ponderación Q y R para sintonizar el comando `lqr()` [19].
- **Asignación de polos:** Se fija la dinámica en bucle cerrado del sistema con el comando `place()` [20].

Comando `lqr()` y comando `place()` para la obtención de K

```
%% OBTENCION DE K = [K1, K2, K3 K4]
% 1. Regulador Lineal Cuadrático (LQR)
Q = diag([1, 1, 1, 1]);
R = 0.1;
[K_lqr, S, E] = lqr(A, B, Q, R);

% 2. Asignación de Polos (Pole Placement)
polos = [-2.5, -3, -3.5, -4];
K_plc = place(A, B, polos);
```

Levantamiento de péndulo

El siguiente hito en el trabajo, fue conseguir levantar el péndulo con la técnica de *Swing-Up* abordada de forma teórica en la Sección 2.3.3. Para conseguirlo se utilizó la misma metodología que para el control de posición del péndulo, se tiene un conjunto de bloques que representan la dinámica del sistema y un bloque para el controlador.

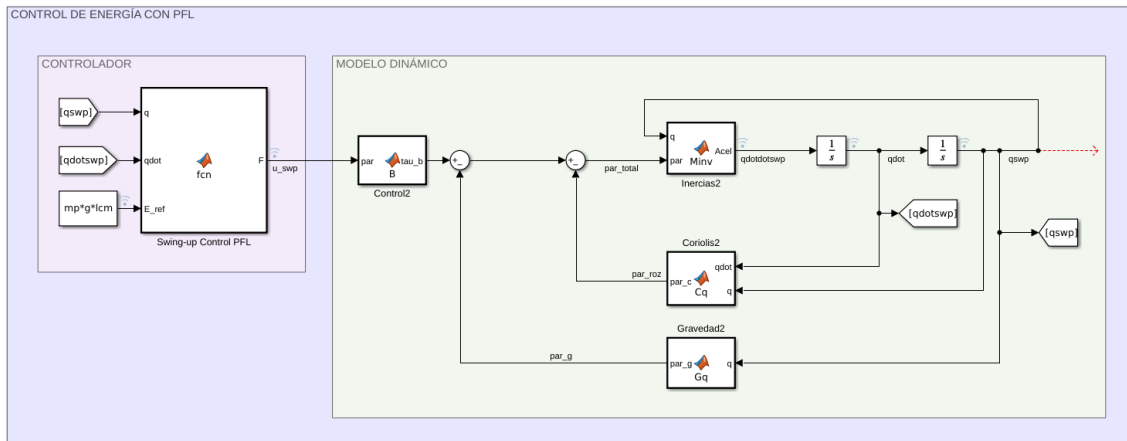


Figura 3.3: Diagrama de bloques con control de energía para el levantamiento del péndulo

Atendiendo a la Figura 3.3 el bloque de simulación tiene dos subconjuntos claramente diferenciados:

- **Control de energía (Zona Izquierda):** Se utiliza una *MATLAB Function* para integrar de forma compacta la técnica de Swing-Up con PFL, tiene como entradas los estados del sistema ($q, \dot{q}$) y la energía de referencia (E_{ref}), como salida proporciona una acción de control (u) en forma de fuerza de empuje horizontal. Para conocer la implementación del controlador en más detalle consulte el Anexo C.
- **Modelo Dinámico (Zona Derecha):** Representa la planta del sistema con los parámetros que la caracterizan. Los detalles de la implementación y el bloque de código de cada función se abordan en la Sección 3.1.1 y el Anexo B.

Conmutación entre controladores

Una vez implementados ambos controladores por separado, para finalizar con la implementación en *Simulink* se aborda la lógica de conmutación explicada en la Sección 2.3.6.

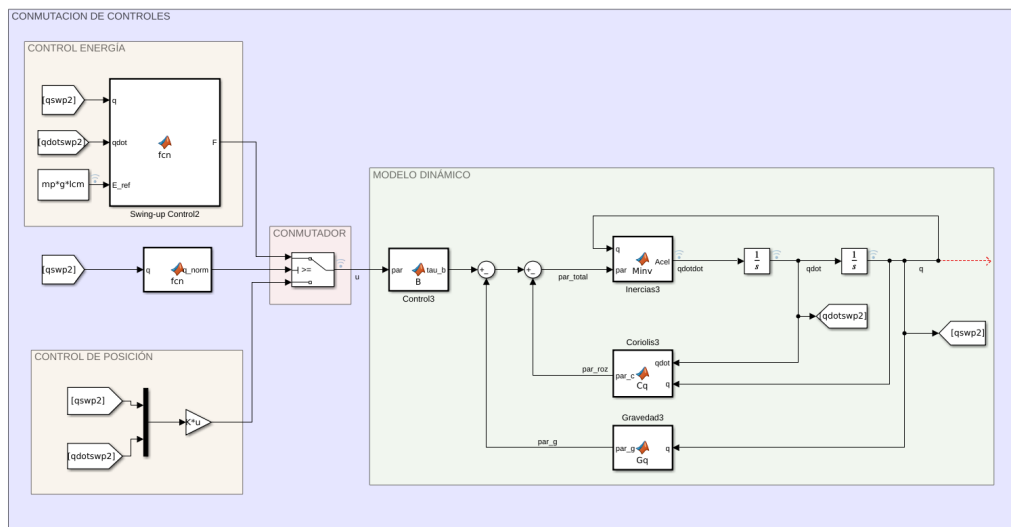


Figura 3.4: Diagrama de bloques conmutación entre controles

La arquitectura implementada en *Simulink* (Figura 3.4) representa el hito central del desarrollo de la simulación. Su diseño permite emular el comportamiento físico del péndulo e integrar de forma segura las estrategias de control antes de su transferencia al hardware real. Para facilitar su comprensión, el diagrama se divide en tres bloques funcionales:

- **Modelo Dinámico de la Planta (Zona Derecha):** Tal y como se comentó durante el capítulo representa el comportamiento mecánico del sistema.
- **Algoritmos de Control (Zona Izquierda):** Resuelven el desafío de control mediante dos estrategias diferenciadas según el estado del péndulo:
 - **Control de Energía (Swing-up):** Actúa cuando el péndulo está hacia abajo, balanceándolo de un lado a otro para inyectarle la energía necesaria para elevarlo.
 - **Control de Posición:** Actúa mediante realimentación de estados para mantener el péndulo en equilibrio estricto una vez ha alcanzado la posición vertical inestable.
- **Lógica de Conmutación (Zona Central):** Es el componente que resuelve la transición automática entre ambos controladores. El bloque `q_norm` normaliza el ángulo del péndulo entre $[-\pi, \pi]$; si este se encuentra lejos de la vertical, el interruptor (`Switch`) selecciona el control de energía. En el instante en que el péndulo entra en la zona de equilibrio (supera cierto umbral angular), el dispositivo conmuta automáticamente al control de posición.

Tras la integración de la lógica de conmutación de ambos controles, la parte de cálculo numérico para las *EDOs* queda resuelta, se pueden ajustar controladores y observar el comportamiento del sistema únicamente con esta implementación. No obstante, atendiendo a la Figura 2.9 sobre la *Arquitectura de la Simulación* falta implementar la conexión con el entorno de *ROS* para la visualización del gemelo digital en *RViz*.

3.1.3 Comunicación con ROS y Visualización del Gemelo Digital

Una vez resuelto el cálculo dinámico y el control en Simulink, el siguiente hito fundamental consistió en establecer la infraestructura de comunicación con el ecosistema de ROS. Esta integración permite materializar el concepto de gemelo digital, facilitando la visualización tridimensional del sistema en tiempo real.

Para articular esta conexión se empleó la herramienta *ROS Toolbox*, estructurando el flujo de información bajo el paradigma de publicador/suscriptor:

- **Lógica de Publicación (Nodo Publicador):** Como se ilustra en la Figura 3.5, el nodo desarrollado en Simulink toma de forma continua el vector con las posiciones x y θ ($[q]$) y el tiempo de simulación para estructurar el envío de datos mediante un bloque de conversión (MATLAB to ROS). Esta información se encapsula en la estructura de un mensaje dedicado a articulaciones (`sensor_msgs/JointState`) y el bloque Publish la difunde activamente en la red bajo el tópico `/joint_states`.

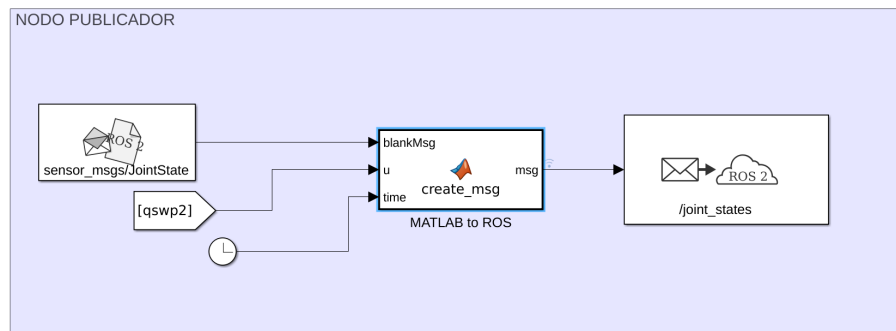


Figura 3.5: Nodo publicador en Simulink

- **Modelado y Visualización en RViz:** Se diseñó un archivo *URDF* que define la geometría y las articulaciones mecánicas del carro y el péndulo. La herramienta de visualización *RViz* se suscribe al tópico generado por Simulink, traduciendo instantáneamente cada mensaje numérico recibido en movimiento visual sobre el modelo 3D.

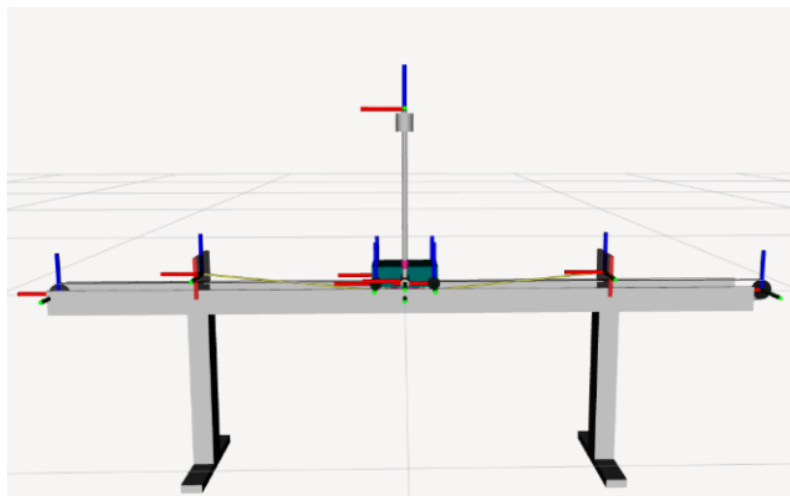


Figura 3.6: Visualización del modelo 3D en RViz

Durante la integración de esta arquitectura, surgieron dos problemas técnicos que requirieron desviaciones de la configuración por defecto para estabilizar la plataforma de simulación:

- **Incompatibilidad en el Middleware (DDS):** Inicialmente se presentaron cortes intermitentes en la comunicación y pérdida de paquetes entre plataformas. El problema se resolvió modificando el estándar de comunicación de red hacia *Cyclone DDS*, lo que garantizó un intercambio de datos robusto y compatible entre MATLAB y ROS. Suele ser un problema muy común en la distribución *Humble* que se empleó para el proyecto.
- **Sincronización del Solver de Simulink:** El motor de cálculo numérico de Simulink tendía a desfasarse de la frecuencia de actualización que exige ROS para la publicación de sus tópicos. Para solucionarlo, se configuró el *solver* en un paso de tiempo fijo discreto.

Si se desea conocer en detalle la configuración de *Simulink* así como la conversión de un vector numérico en un mensaje estándar de ROS, consulte el Anexo D.

3.1.4 Simulaciones Recursivas

Para finalizar el bloque de simulación, se desarrolló una arquitectura alternativa basada en la programación estructurada de un bucle iterativo, prescindiendo de dependencias externas de entornos de simulación comerciales. El desarrollo de este entorno de simulación "en código" responde a dos objetivos fundamentales:

- **Portabilidad:** El código fuente es reproducible y ejecutable en cualquier plataforma. Aunque se implementó inicialmente en MATLAB para mantener la integración con el resto del proyecto, esta estructura facilita su futura migración a nodos de ROS en lenguajes como C++ o Python.
- **Transición natural hacia la implementación real:** La estructura recursiva del código emula fielmente el bucle de control de un microcontrolador real (que trabaja a intervalos de tiempo fijos), simplificando el paso del software al hardware definitivo.

Para lograr esto de forma autónoma, se sustituyó el motor de física comercial por un algoritmo de integración numérica elemental (método de Euler de primer orden), los valores de los estados ($x_k = [x, \theta, \dot{x}, \dot{q}]^T$) en la siguiente iteración del bucle vienen dados por la expresión recursiva:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + T_s \cdot f(\mathbf{x}_k, u_k) \quad (3.1)$$

Esta ecuación gobierna el comportamiento del simulador en cada paso de tiempo y se compone de los siguientes elementos:

- $\mathbf{x}_{k+1}$: Representa el **estado futuro** del sistema (posiciones y velocidades que tendrá el mecanismo en el siguiente instante).
- $\mathbf{x}_k$: Es el **estado actual** conocido en la iteración presente.
- T_s : Es el **periodo de muestreo**, es decir, el pequeño intervalo de tiempo que transcurre entre cada iteración del bucle.
- $f(\mathbf{x}_k, u_k)$: Es la función que describe la **dinámica física del sistema**. Combina el estado actual $\mathbf{x}_k$ con la acción de control en ese momento (u_k) para devolver las velocidades y aceleraciones instantáneas del mecanismo.

Mientras que las velocidades se obtienen de forma directa, el cálculo de las aceleraciones dentro de la función $f(\mathbf{x}_k, u_k)$ requiere resolver algebraicamente las ecuaciones que definen la dinámica del sistema abordadas en la Sección 2.3.2. El desarrollo matemático completo para resolver este sistema dinámico y su traducción algorítmica al código de simulación se detallan en el Apéndice E.

Problemas Encontrados y Compromiso del Periodo de Muestreo

El principal reto técnico durante el desarrollo de este entorno simulado fue la aparición de **inestabilidades numéricas artificiales**. El éxito de la simulación depende críticamente de la elección del periodo de muestreo (T_s).

Debido a que el método de Euler asume de forma simplificada que las aceleraciones del sistema se mantienen constantes durante todo el intervalo T_s , se detectó que valores de tiempo excesivamente grandes entre iteraciones acumulaban un error de truncamiento que no correspondía con la realidad física.

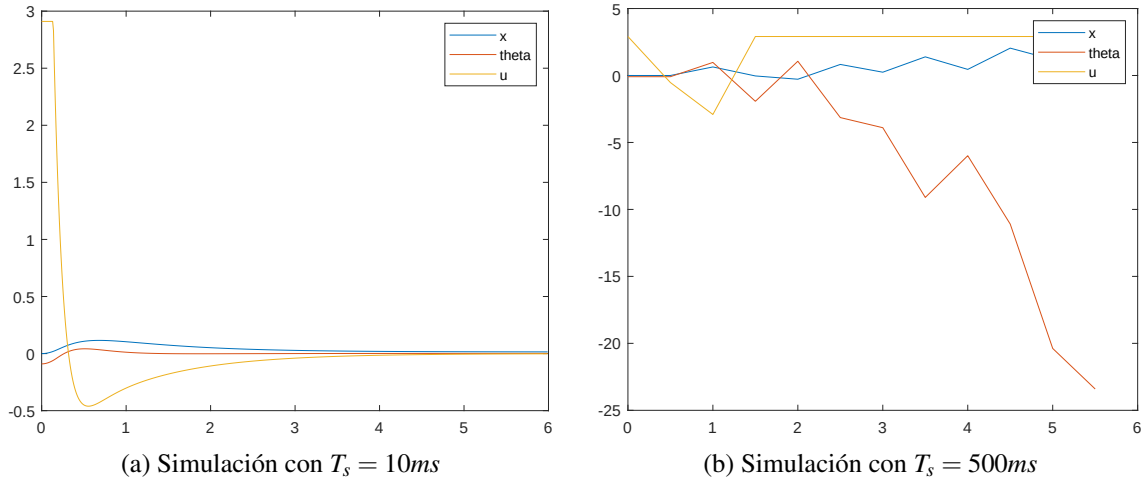


Figura 3.7: Dependencia de T_s en la simulación con expresiones recursivas

Para concluir, la Figura 3.7 ilustra el impacto crítico del periodo de muestreo al aplicar el mismo control de posición por realimentación del estado. En ella se observa cómo un intervalo de 10 ms produce una respuesta estable y fiel al comportamiento real del sistema, mientras que elevar dicho periodo a 500 ms degrada por completo la convergencia numérica, provocando una respuesta oscilatoria e inestable que distorsiona la física del mecanismo.

3.2 Programación del microcontrolador

En esta sección se detalla el desarrollo e implementación del firmware encargado de ejecutar las tareas de control en tiempo real. En primer lugar, se describe de forma breve el ecosistema de desarrollo empleado, la asignación física de los pines (PINOUT) y la forma general de configurar los periféricos involucrados.

Posteriormente, se desglosa con mayor detalle la configuración a bajo nivel de los periféricos críticos de actuación (ePWM) y captura de datos (eQEP), para finalizar con el diseño del canal de telemetría (SCI) y la sincronización de la interrupción periódica del sistema (rutina de control).

Ecosistema de desarrollo

El desarrollo del firmware, como se comentó en el Capítulo 2, se apoya en el entorno de desarrollo integrado (IDE) oficial de Texas Instruments, **Code Composer Studio (CCS)** [11]. Este entorno optimiza la compilación para la arquitectura C28x y proporciona herramientas avanzadas de depuración (*debug*). Gracias a ellas, es posible ejecutar el código línea por línea, así como inspeccionar y monitorizar el valor de los registros del sistema en tiempo real durante la ejecución del programa.

Además para programar el microcontrolador **TMS320F280049C** en el IDE, se tiene la SDK **C2000Ware** que ofrece tres metodologías de acceso a los periféricos y registros de memoria, cada una con un compromiso (*trade-off*) diferente entre nivel de abstracción, legibilidad y control sobre la programación: el acceso directo por dirección, el mapeo mediante estructuras de campos de bits (*Bit-fields*) y el uso de la librería de funciones *DriverLib* [21].

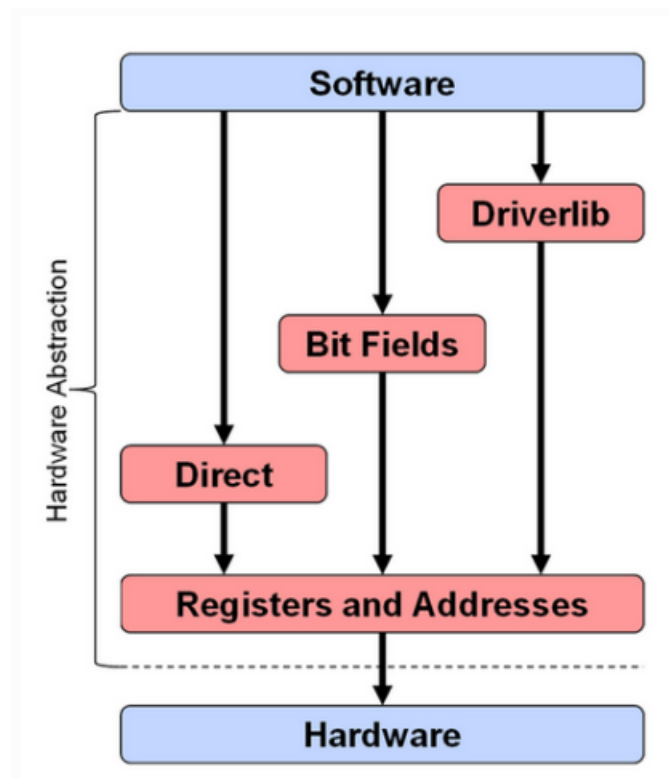


Figura 3.8: Niveles de abstracción para programar el microcontrolador [21]

El **Acceso Directo** ofrece un control absoluto pero con una dificultad de desarrollo muy alta y propensión a errores, al obligar a calcular manualmente máscaras binarias en hexadecimal. En el extremo opuesto, la librería **DriverLib** reduce la dificultad al mínimo mediante funciones abstractas de alto nivel, pero resta control directo al desarrollador sobre la configuración real de los registros del chip.

Por este motivo, se ha seleccionado el enfoque intermedio de **Campos de bits (Bit-fields)**. Esta metodología permite mapear los registros físicos del *datasheet* como estructuras de datos nativas en C. De este modo, se justifica su uso al garantizar un **control directo e inmediato sobre el hardware** bajo una dificultad moderada.

Cuadro 3.2: Comparativa de metodologías

Metodología	Dificultad	Control del Hardware	Ejemplo de Sintaxis
Acceso Directo	Alta	Absoluto (Manual)	*CMPR1 = 0x1234;
Bit-fields	Media	Alto (Directo)	EPwm1Regs.CMPA.bit.CMPA = valor;
DriverLib	Baja	Bajo (Abstracto)	EPWM_setCounterCompareValue()

La principal ventaja práctica de esta elección se manifiesta durante las etapas de depuración. Al haber programado utilizando los nombres nativos de los registros, el mapa de variables que despliega el IDE es plenamente inteligible y mantiene una trazabilidad directa con el código fuente; un beneficio que se perdería drásticamente si se empleara la abstracción de *DriverLib*.

Como se muestra en la Figura 3.9, durante la depuración del programa `control.c`, el entorno de desarrollo permite inspeccionar y monitorizar en tiempo real el valor exacto de los bits y registros críticos del sistema.

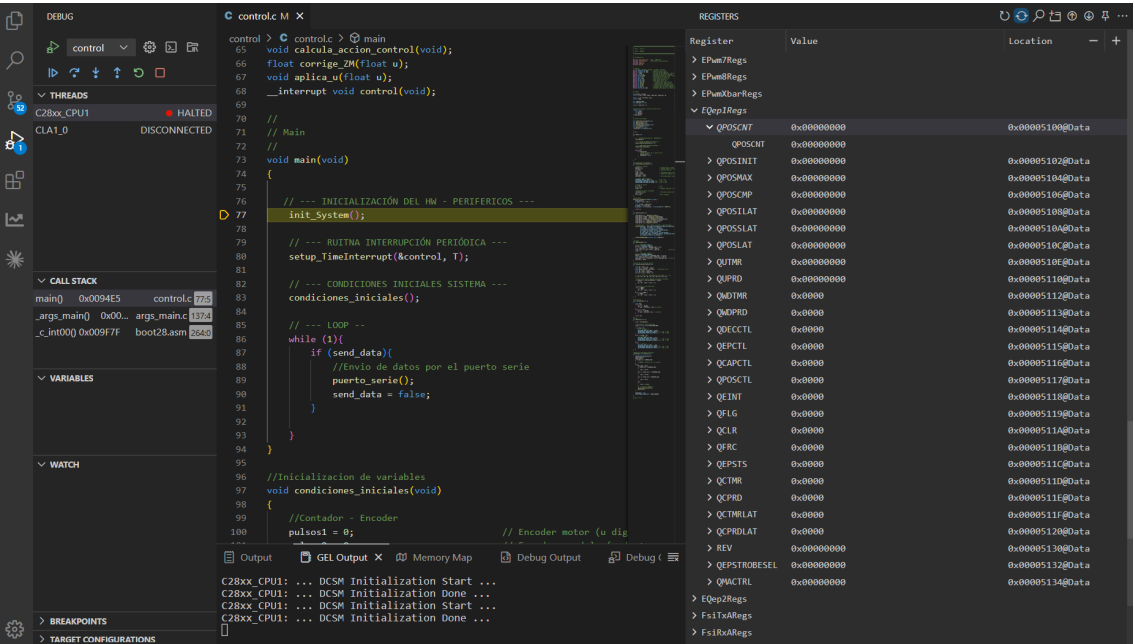


Figura 3.9: Depuración de código con CCS

Mapeo de periféricos y asignación de pines (PINOUT)

Antes de proceder con el desarrollo software, se define la interfaz física del sistema (*PINOUT*). El microcontrolador **TMS320F280049C** cuenta con un mecanismo interno de multiplexado para optimizar el uso de sus pines físicos, es decir, un mismo pin físico puede enrutarse internamente hacia módulos digitales independientes, entradas analógicas o funciones de propósito general (GPIO). La tabla completa de multiplexado del chip se proporciona en el Anexo F. Para la realización de este proyecto se emplea la tarjeta de desarrollo oficial **LAUNCHXL-F280049C**. La distribución espacial y función de los pines se ilustra en la Figura 3.10.

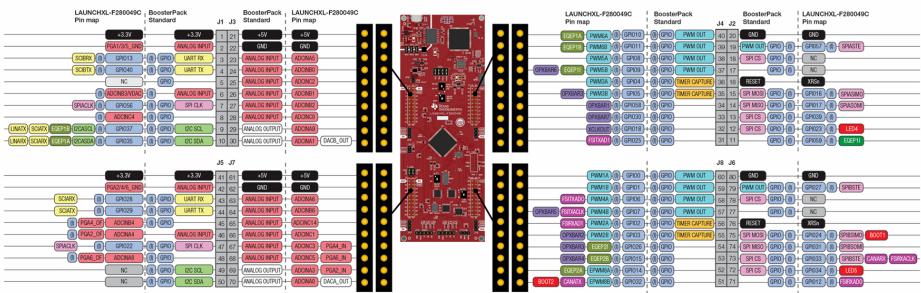


Figura 3.10: Mapa de pines y asignación de periféricos de la tarjeta LAUNCHXL-F280049C.

A partir de la distribución espacial provista por el fabricante y considerando los requerimientos operativos (actuación del motor, lectura de encoders y comunicación externa), se seleccionan los pines atendiendo a funcionalidad y localización en la tarjeta de desarrollo. En la Tabla 3.3 se consolida el mapeo definitivo de pines

Cuadro 3.3: Mapeo definitivo de entradas y salidas

Señal Física	Conexión LaunchPad	GPIO	Mux Value	Descripción Técnica / Propósito
EPWM1A	Pin 80 (J8)	GPIO0	1 (<i>ePWM</i>)	Señal de modulación de potencia (<i>ENA</i> del puente H).
MOTOR_IN1	Pin 79 (J8)	GPIO1	0 (<i>GPIO</i>)	Control de dirección del motor (<i>IN1</i> del puente H).
MOTOR_IN2	Pin 78 (J8)	GPIO6	0 (<i>GPIO</i>)	Control de dirección del motor (<i>IN2</i> del puente H).
EQEP1A	Pin 40 (J4)	GPIO10	5 (<i>eQEP</i>)	Entrada del Canal A para el encoder del carro.
EQEP1B	Pin 39 (J4)	GPIO11	5 (<i>eQEP</i>)	Entrada del Canal B para el encoder del carro.
EQEP2A	Pin 72 (J8)	GPIO14	10 (<i>eQEP</i>)	Entrada del Canal A para el encoder del péndulo.
EQEP2B	Pin 73 (J8)	GPIO15	10 (<i>eQEP</i>)	Entrada del Canal B para el encoder del péndulo.
SCIRXDA	Enrutamiento interno ¹	GPIO28	1 (<i>SCI</i>)	Entrada de recepción de comandos desde el PC.
SCITXDA	Enrutamiento interno ¹	GPIO29	1 (<i>SCI</i>)	Salida de transmisión de telemetría hacia el PC.

¹Las señales del módulo SCI (UART) se encuentran interconectadas por hardware en el circuito de la placa LAUNCHXL-F280049C hacia el depurador XDS110. Esto permite la comunicación bidireccional con el PC a través del puerto JTAG con un único cable USB.

Configuración general de periféricos hardware dedicados (ePWM y eQEP)

La utilidad de este apartado radica en exponer el procedimiento que se ha diseñado en el firmware para la correcta habilitación, inicialización y puesta en marcha de los periféricos esenciales del sistema. Así, se desarrollaron las rutinas de configuración tanto para el módulo de modulación por ancho de pulsos (*ePWM1*) —encargado de la actuación sobre el motor— como para los bloques de decodificación en cuadratura (*eQEP1* y *eQEP2*) dedicados a la lectura de los encoders. Con el fin de estructurar el código de manera eficiente, el proceso de inicialización de cada módulo consta de tres fases fundamentales:

- **Habilitación del reloj del periférico:** Se activa la señal de reloj específica a través de los registros de control del sistema (`CpuSysRegs.PCLKCR2` para el módulo *ePWM1* y `CpuSysRegs.PCLKCR4` para los módulos *eQEP*). Debido a que estos registros están protegidos, se requiere el uso de las instrucciones de ensamblador `EALLOW` y `EDIS` para su modificación segura.
- **Configuración de registros internos:** Ajuste de los registros del módulo de acuerdo con las especificaciones del fabricante para obtener el comportamiento deseado. Para facilitar la lectura del texto principal, la descripción exhaustiva del mapa de memoria de cada bloque se ha desplazado al Anexo G (*eQEP*) y al AnexoH (*ePWM*).
- **Activación final del módulo:** Modificación del bit de habilitación global por hardware (como el bit `OPEN` en el caso de la lógica *eQEP*) para indicar a la CPU que el periférico se encuentra plenamente operativo.

Dada la relevancia de estos submódulos, en los siguientes apartados se analiza con detalle su implementación y funcionamiento interno.

3.2.1 Módulo eQEP (Lectura del Encoder)

La lectura de los encoders ópticos se delega en una máquina de estados integrada en el hardware del periférico *eQEP*. Como se ilustra en la Figura 3.11, las señales físicas de los canales *QEPA* y *QEPB* presentan un desfase eléctrico de 90° . Al activar el modo de decodificación en cuadratura, el hardware detecta cada flanco de subida y bajada, lo que permite transitar por cuatro combinaciones binarias distintas (00, 10, 11, 01) por cada periodo de la señal.

Esta decodificación no solo cuadriplica ($\times 4$) la resolución del encoder, sino que permite discernir instantáneamente el sentido de giro (horario o antihorario) según la secuencia de transición de los estados.

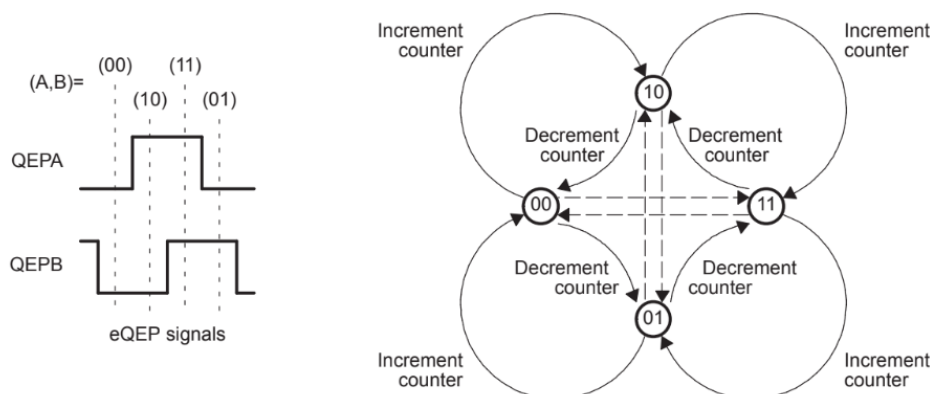


Figura 3.11: Máquina de estados, decodificación en cuadratura [22]

En la Tabla 3.4 se detalla la configuración final asignada a los campos de bits de este módulo para conseguir la decodificación en cuadratura. Si se desea conocer en detalle el mapa de memoria y configuraciones alternativas consulte el Anexo ??.

Cuadro 3.4: Configuración final de los registros del módulo eQEP

Registro / Campo	Valor Asignado	Justificación Técnica / Propósito
QDECCTL.bit.QSRC	0	Modo cuadratura (resolución $\times 4$).
QEPCTL.bit.FREE_SOFT	2	El contador ignora los puntos de parada del <i>debugger</i> .
QPOSMAX	0xFFFFFFFF	Límite máximo de conteo (rango completo de 32 bits).
QPOSINIT	0	Posición de inicialización.
QPOSCNT	0	Reset del contador de posición al arrancar.
QEPCTL.bit.QPEN	1	Habilitación del periférico.

La asignación de un rango de 32 bits mediante QPOSMAX garantiza que el sistema pueda registrar el desplazamiento total del carro y la posición del péndulo sin riesgo de desbordamiento (*overflow*). Además, la configuración actual del campo FREE_SOFT está pensada para las fases de depuración: al indicarle al periférico que ignore los puntos de parada del entorno de desarrollo, se evita que el contador de posición se congele o pierda la sincronía con el estado físico real del sistema cuando se pausa manualmente la ejecución del programa.

3.2.2 Módulo ePWM1 (Actuación del Motor)

Para comprender el funcionamiento del periférico *ePWM*, es necesario analizar tanto su infraestructura interna como el comportamiento de su temporizador principal. En la Figura 3.12 se presenta el diagrama de bloques general del módulo. Como se observa, el periférico no es un simple temporizador, sino un ecosistema compuesto por varios submódulos especializados.

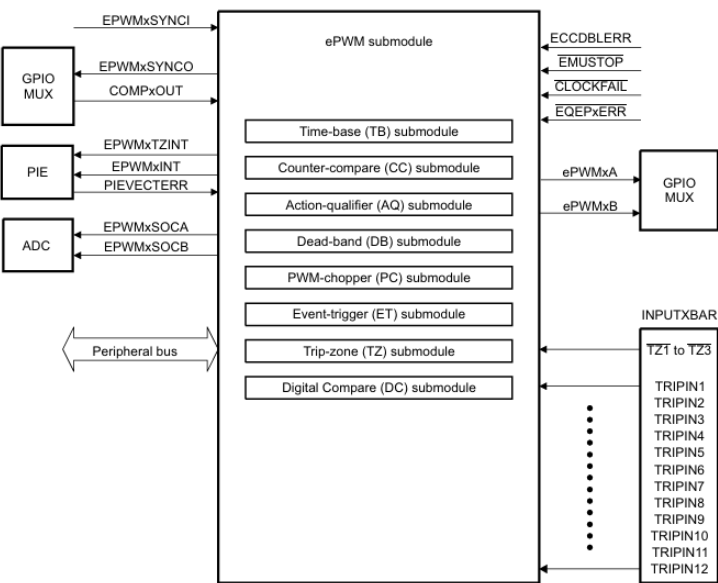


Figura 3.12: Diagrama de bloques y submódulos del periférico ePWM [22].

El comportamiento de la señal de modulación se rige principalmente por la acción combinada de sus tres primeros submódulos:

- **Time-base (TB) submodule**
- **Counter-compare (CC) submodule**
- **Action-qualifier (AC) submodule**

El **Submódulo Time-base** determina la simetría, la resolución y la frecuencia de la señal PWM. Como ilustra la Figura 3.13, se ha seleccionado el modo simétrico triangular o conteo hacia arriba y hacia abajo (*Up-Down count*). En este modo, el contador dinámico TBCTR incrementa su valor hasta alcanzar el límite cargado en el registro de periodo TBPRD y luego desciende simétricamente hasta cero, duplicando la duración del periodo de la señal PWM respecto a los modos unidireccionales.

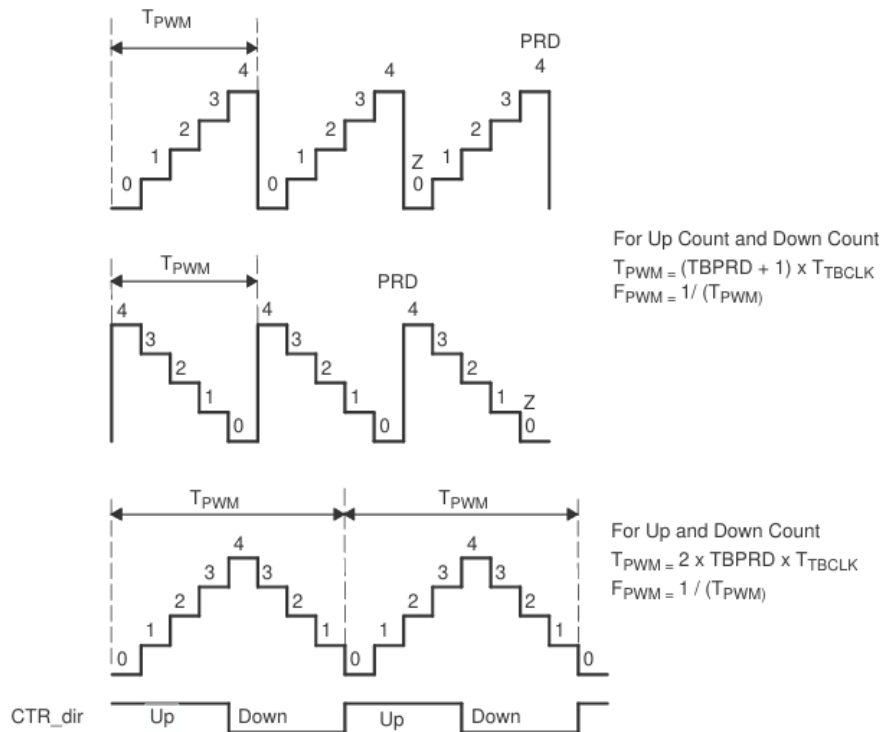


Figura 3.13: Modos de conteo del submódulo Time-base (TB) y ecuaciones de periodo [22].

A partir de la rampa triangular establecida, el **Submódulo de Comparación (Counter-compare)** supervisa en tiempo real el valor instantáneo del contador TBCTR y lo contrasta con el registro CMPA.

La generación real de la onda cuadrada se delega posteriormente en el **Submódulo Calificador de Acción (Action-qualifier)**. Cuando TBCTR intersecta el valor de CMPA, el periférico altera instantáneamente el estado del pin físico sin intervención de la CPU, forzando un nivel lógico bajo (LOW) durante la fase de subida (*Up-count*) y un nivel alto (HIGH) durante la fase de bajada (*Down-count*).

Debido a esta combinación de eventos implementada en el firmware, el ciclo de trabajo (*duty cycle*) de la señal resultante guarda una relación lineal y directamente proporcional con el valor cargado en CMPA:

- **CMPA = 0:** Las intersecciones ocurren en el origen de la rampa. Al iniciar la pendiente ascendente, el evento de subida (CAU) fuerza de inmediato la señal a nivel bajo, manteniéndola de forma continua en 0 V, lo que equivale a un **ciclo de trabajo del 0 %** (motor completamente detenido).
- **CMPA = TBPRD / 2:** Las intersecciones ocurren exactamente en el punto medio de ambas pendientes, generando una onda perfectamente simétrica con un **ciclo de trabajo del 50 %** (el motor recibe la mitad de la tensión eficaz).
- **CMPA = TBPRD:** El umbral de comparación se sitúa en el vértice superior de la rampa triangular, maximizando el tiempo en que la señal permanece en nivel lógico alto. Esto se traduce en un **ciclo de trabajo del 100 %** (el motor opera a su máxima velocidad de giro al recibir el valor eficaz completo de la fuente).

En la Tabla 3.5 se sintetiza la configuración definitiva cargada en los campos de bits de la instancia EPwm1Regs para establecer una frecuencia de conmutación de 1 kHz con conteo simétrico.

Cuadro 3.5: Configuración final de los registros de control del módulo ePWM1.

Registro / Campo	Valor Asignado	Justificación Técnica / Propósito
TBPRD	5000	Resolución para modular el ciclo de trabajo
TBCTR	0	Inicialización y limpieza preventiva del contador de tiempo base.
TBCTL.bit.CTRMODE	2	Configuración de conteo triangular simétrico.
TBCTL.bit.HSPCLKDIV	5	Pre-escalador de reloj, alta velocidad $TBCLK = SYSCLK / (HSPCLKDIV \cdot CLKDIV)$.
TBCTL.bit.CLKDIV	1	Pre-escalador de reloj secundario.
CMPA.bit.CMPA	0	Ciclo de trabajo (<i>duty cycle</i>) inicial en un estado seguro del 0 %.
AQCTLA.bit.CAU	1	Fuerza la señal a 0 V al cruzar el umbral CMPA en la fase de subida.
AQCTLA.bit.CAD	2	Fuerza la señal a 3.3 V al cruzar el umbral CMPA en la fase de bajada.

Compensación y caracterización de la zona muerta

Un desafío crítico en el control del motor de corriente continua mediante el puente en H es la presencia de la **zona muerta** (*dead-band*). Debido a los tiempos de conmutación físicos e inercias eléctricas de los transistores Mosfet o transistores de potencia del driver, existe un rango mínimo de tensión (ciclo de trabajo muy bajo) en el cual la corriente aplicada no es capaz de vencer la fricción ni activar el movimiento, rompiendo la linealidad del actuador en las inmediaciones del cero.

Para mitigar este efecto no lineal y maximizar la respuesta en baja velocidad del carro, se aplicaron dos técnicas de diseño coordinadas:

1. **Ajuste empírico de la frecuencia PWM:** Al consolidar la frecuencia en 10 kHz, se balancea el compromiso dinámico. Frecuencias excesivamente altas empeoran los efectos de la zona

muerta por las pérdidas en conmutación, mientras que frecuencias muy bajas introducen oscilaciones indeseadas en el par motor.

2. **Algoritmo de compensación por software:** Se implementó una rutina en el bucle cerrado encargada de monitorizar el signo de la acción de control calculada. Si la ley de control exige un valor absoluto inferior al umbral mínimo caracterizado de la zona muerta, el software inyecta un escalón de pre-activación constante (*offset*) que desplaza el punto de operación dinámico directamente hacia la zona lineal del driver, eliminando el estancamiento del sistema.

3.2.3 Protocolo de comunicación serie y telemetría (SCI)

3.2.4 Rutina de interrupción periódica

1)Firmware, comentar como se programa una DSP de texas instruments mediante el uso del IDE CCS y la capa de HAL C2000-ware. Utilizar el datashet del fabricante para justificar la configuración de ciertos registros. Módulos Epwm y Eqp muy interesante ver como liberan a la CPU de estas tareas. Explicar la configuración de registros para conseguir cierto comportamiento en los periféricos así como explicar como se configuró el protocolo serie para la telemetría del sistema.

2)Explicar como se programo la DSP con timers para ejecutar el bucle de control en una rutina de interrupción periódica (implica determinismo, nada del loop de Arduino).

3.3 Identificación de parámetros

4)Razonar experimentos de identificación de parámetros del sistema. La justificación es que los parámetros del fabricante no sabemos si son del todo ciertos al ser una máquina de hace 25 años. Algunos experimentos: Conversión de voltios a fuerza horizontal (encontrar la constante que relaciona ambas magnitudes), Identificación del rozamiento horizontal del carro, identificación del rozamiento y la inercia del péndulo.

5)Simular de nuevo con los parámetros obtenidos mediante identificación.

3.4 Implementación del algoritmo de control

6)Implementación de algoritmos de control en el microcontrolador (lo tenemos todo dividido en funciones, explicar un poco que recibe y devuelve cada una y como se ajusta todo)

4. Resultados

- 4.1 Resultados en simulación
- 4.2 Resultados en sistema físico

5. Conclusiones y posible trabajo futuro

Trabajo a futuro → Implementación en HAL, uso de algoritmos no basados en modelo (Reinforcement Learning)

Bibliografía

- [1] Feedback Instruments Ltd., *Digital Pendulum System: Getting Started (33-005-1M5)*, MATLAB 5 Version. Manual Part No. 1160-330051M5, Feedback Instruments Ltd., Crowborough, UK, 1998 (véase página 6).
- [2] Feedback Instruments Ltd., *Digital Pendulum System: External Interface (33-005-3M5)*, Explains the 16-bit DLL libraries and Real-Time Kernel for Windows 95, Feedback Instruments Ltd., Crowborough, UK, 1998 (véase página 6).
- [3] M. W. Spong, S. Hutchinson y M. Vidyasagar, *Robot modeling and control*. John Wiley & Sons, 2006 (véase página 14).
- [4] R. M. Murray, Z. Li y S. S. Sastry, *A mathematical introduction to robotic manipulation*. CRC press, 1994 (véase página 14).
- [5] R. Tedrake, *Underactuated Robotics: Algorithms for Planning and Control*. MIT Press, 2023, Disponible online en <http://underactuated.mit.edu/> (véanse páginas 15, 19).
- [6] K. J. Åström y K. Furuta, "Swinging up a pendulum by energy control," *Automatica*, volumen 36, páginas 287-295, 2000, Disponible online en <https://www.elsevier.com/locate/automatica> (véase página 17).
- [7] M. W. Spong, "Partial Feedback Linearization of Underactuated Mechanical Systems," en *Proceedings of the 1994 IEEE International Conference on Robotics and Automation*, Coordinated Science Laboratory, University of Illinois at Urbana-Champaign, IEEE, San Diego, CA, mayo de 1994, páginas 2356-2361 (véase página 18).
- [8] I. P. Alós, *Tema 7 - Análisis en Espacio de Estados del comportamiento dinámico de sistemas*, Apuntes de la asignatura IR2124 Sistemas Automáticos Avanzados, Grado en Inteligencia Robótica, Universitat Jaume I de Castelló, 2025 (véanse páginas 20, 21).
- [9] I. P. Alós, *Tema 8 - Control por realimentación del estado*, Apuntes de la asignatura IR2124 Sistemas Automáticos Avanzados, Grado en Inteligencia Robótica, Universitat Jaume I de Castelló, 2025 (véanse páginas 22, 23).

- [10] *TMS320F28004x Real-Time Microcontrollers Data Sheet*, manual, Literature Number: SPRSAD2J. Disponible en <https://www.ti.com/lit/ds/symlink/tms320f280049c.pdf>, Texas Instruments, 2024 (véase página 26).
- [11] T. Instruments, *Code Composer Studio (CCS) Integrated Development Environment*, misc, Disponible en <https://www.ti.com/tool/CCSTUDIO>, 2025 (véanse páginas 26, 39).
- [12] T. Instruments, *C2000Ware Software Development Kit*, misc, Disponible en <https://www.ti.com/tool/C2000WARE>, 2025 (véase página 26).
- [13] *7A-160W Dual Channel DC Motor Drive Module User Manual*, manual, Modelo: AQMH2407ND. Disponible en <https://www.handsontec.com/dataspecs/module/7A-160W%20motor%20control.pdf>, HandsOn Technology, 2024 (véase página 27).
- [14] *L298 Dual Full-Bridge Driver Data Sheet*, manual, Disponible en <https://www.st.com/resource/en/datasheet/l298.pdf>, STMicroelectronics, 2000 (véase página 27).
- [15] *EL357N-G Series: 4 PIN SOP Phototransistor Photocoupler*, manual, Literature Number: DPC-0000014 Rev. 6, Everlight Electronics Co., Ltd., 2014 (véase página 27).
- [16] *HEDS-9040/9140 Series Three Channel Optical Incremental Encoder Modules Data Sheet*, manual, Documento: HEDS-9140-DS100. Disponible en <https://www.broadcom.com/products/motion-control-encoders/incremental-encoders/optical-encoder-modules/heds-9140-a00>, Broadcom (Avago Technologies), 2024 (véase página 28).
- [17] *D.C. Direct Drive Motors: Type 82 760 0*, manual, Disponible en <https://www.crouzet.com/>, Crouzet Automatismes, 2024 (véase página 29).
- [18] Feedback Instruments Ltd., *Digital Pendulum System: Teaching Manual (Model 33-005-4M5)*, Edition 01. Part No. 1160-330054M5, Feedback Instruments Ltd., Crowborough, East Sussex, UK, 1998 (véase página 32).
- [19] MathWorks, *Linear-Quadratic Regulator (LQR) design - MATLAB lqr*, misc, Disponible online en la documentación oficial de Control System Toolbox, 2026. dirección: <https://es.mathworks.com/help/control/ref/lti.lqr.html> (véase página 33).
- [20] MathWorks, *Pole placement design - MATLAB place*, misc, Disponible online en la documentación oficial de Control System Toolbox, 2026. dirección: <https://es.mathworks.com/help/control/ref/place.html> (véase página 33).
- [21] Texas Instruments, *C2000Ware Peripheral Driver Library User's Guide*, 2024. dirección: https://software-dl.ti.com/C2000/docs/software_guide/c2000ware/drivers.html (véase página 39).
- [22] Texas Instruments, *TMS320F28004x Real-Time Microcontrollers Technical Reference Manual*, SPRUI33H, Disponible en línea., Texas Instruments, Dallas, Texas, 2024. dirección: <https://www.ti.com/lit/ug/sprui33h/sprui33h.pdf> (véanse páginas 42-44, 62-67).
- [23] M. Porcar, *TFG_Control_MiguelPorcar: Control de sistemas robóticos subactuados*, https://github.com/MPV247/TFG_Control_MiguelPorcar, Accedido: 19 de mayo de 2026, 2026 (véase página 61).

6. Apéndices

A Desarrollo del Lagrangiano

En este anexo se detalla el desarrollo matemático paso a paso del formalismo de Euler-Lagrange, partiendo de las energías del sistema hasta obtener la formulación matricial.

B Dinámica en MATLAB Functions

Este apéndice agrupa los bloques de código implementados en Simulink mediante estructuras *MATLAB Function* para modelar la dinámica del sistema mecánico de dos grados de libertad.

MATLAB Function $M(q)^{-1}$

```
function Acel = Minv(q, par, J_p, M, m, l)
    M = [M + m,          +m*l*cos(q(2));
         +m*l*cos(q(2)),    J_p];
    Acel = M\par; %Division de matrices.
end
```

Esta función calcula el vector de aceleraciones del sistema a partir de la inversión numérica de la matriz de inercia. Para optimizar el coste computacional, evita el cálculo explícito de la matriz inversa utilizando el operador de división a la izquierda.

Entradas: Vector de posiciones q , vector de fuerzas/pares generalizados remanentes par (derivado de restarle a las fuerzas de control los efectos de fricción, gravedad y Coriolis), inercia del péndulo J_p , masa del carro M , masa del péndulo m , y distancia al centro de masas l .

Salidas: Vector de aceleraciones generalizadas $\ddot{q} = [\ddot{x}, \ddot{\theta}]^T$.

Operación: Resuelve algebraicamente el sistema lineal para obtener $\ddot{q} = M(q)^{-1} \cdot \tau_{neto}$.

MATLAB Function $C(\dot{q}, q)$

```
function par_c = Cq(qdot, q, cx, ctheta, m, l)
    C = [cx,      -m*l*qdot(2)*sin(q(2))
         0,        ctheta                ];
    par_c = C*qdot;
end
```

Esta función evalúa los efectos dinámicos asociados a las fuerzas centrífugas, de Coriolis y a los coeficientes de amortiguamiento viscoso presentes en las articulaciones del mecanismo.

Entradas: Vector de velocidades $qdot = [\dot{x}, \dot{\theta}]^T$, vector de posiciones $q = [x, \theta]^T$, coeficientes de fricción viscosa (c_x, c_θ) , masa m y longitud del péndulo l .

Salidas: Vector de fuerzas generalizadas de Coriolis y amortiguamiento $par_c = C(\dot{q}, q)\dot{q}$.

Operación: Construye la matriz no lineal $C(\dot{q}, q)$ y realiza la multiplicación matricial por el vector de velocidades $(\dot{q})$.

MATLAB Function $G(q)$

```
function par_g = Gq(q, m, g, l)
    G = [0; m*g*l*sin(q(2))];
    par_g = G;
end
```

Esta función determina el par generado por el campo gravitatorio sobre el péndulo basándose en su posición angular.

Entradas: Vector de posiciones q , masa del péndulo m , constante de gravedad g , y longitud l .

Salidas: Vector de fuerzas gravitacionales generalizadas $par_g = G(q)$.

Operación: Evalúa el par gravitatorio que afecta exclusivamente al segundo grado de libertad (coordenada angular del péndulo $\theta = q(2)$).

MATLAB Function B

```
function tau_b = B(u)
    tau_b = [1; 0]*u;
end
```

Esta función mapea la acción de control hacia el espacio de coordenadas generalizadas del modelo dinámico (matriz de acoplamiento de la entrada).

Entradas: Señal de control escalar u (fuerza aplicada sobre el carro).

Salidas: Vector de fuerzas generalizadas de entrada $tau_b = B \cdot u$.

Operación: Proyecta la fuerza u sobre el primer grado de libertad (movimiento lineal del carro), asumiendo que el péndulo no tiene ningún actuador.

C Código Control de Energía

MATLAB Function Swing-Up con PFL

```
function F = fcn(q, qdot, E_ref, m, l, ctheta, cx, J_p, M)
    % --- PARAMETROS FISICOS Y GANANCIAS ---
    g = 9.81;
    Ke = 26;
    kp = 6;
    kd = 4;
    % -- ESTADOS DEL SISTEMA --
    x = q(1);           % Posicion del carro
    theta = q(2);        % Angulo del pendulo
    xdot = qdot(1);      % Velocidad del carro
    thetadot = qdot(2);  % Velocidad angular
    % --- 1. CALCULO DE ENERGÍA ---
    Ep = m * g * l * cos(theta);
    Ek = 0.5 * J_p * thetadot^2;
    E_total = Ep + Ek;
    % --- 2. LEY DE CONTROL DE ENERGIA (Aceleracion 'u') ---
    e = E_total - E_ref;
    u = Ke * thetadot * cos(theta) * e - kp * x - kd * xdot;
    % --- 3. LINEARIZACION POR RETROALIMENTACION PARCIAL (PFL)
    % ---
    % Convierte la aceleracion deseada 'u' en Fuerza 'F' para el
    motor
    thetadotdot = 1/J * (-m*l*cos(theta)*u ...
        + m*g*l*sin(theta) - ctheta*thetadot);
    F = (M + m) * u ...
        - (m * l * thetadot^2 * sin(theta)) ...
        + (m * l * cos(theta)) * thetadotdot ...
        + cc*xdot;
    % --- 4. SATURACION ---
    F_max = 2; % Newtons
    if F > F_max
        F = F_max;
    elseif F < -F_max
        F = -F_max;
    end
end
```

Entradas: Vector de posiciones $q = [x, \theta]^T$, vector de velocidades $qdot = [\dot{x}, \dot{\theta}]^T$, energía mecánica de referencia del péndulo en posición vertical E_{ref} .

Salidas: Fuerza de empuje F (en Newtons).

Operación: El algoritmo ejecuta consecutivamente cuatro etapas fundamentales:

1. **Cálculo de Energía:** Evalúa la energía mecánica total instantánea del péndulo ($E_{total} = E_k + E_p$).
2. **Control de Energía:** Genera una aceleración de referencia u proporcional al error de energía (e).
3. **Linealización PFL:** Traduce analíticamente la aceleración virtual u en la fuerza real F requerida para compensar la dinámica no lineal del carro.
4. **Saturación:** Limita la salida a un umbral simétrico (± 2 N).

D Configuración Conexión MATLAB ROS

MATLAB Function conversión a mensaje de ROS

```
function msg = create_msg(blankMsg, q, time, start_time_epoch)
    msg = blankMsg;
    %Sincronizacion con el reloj interno de la máquina.
    total_time = start_time_epoch + time;.
    s = floor(total_time);
    ns = (total_time - s) * 1e9;
    msg.header.stamp.sec = int32(s);
    msg.header.stamp.nanosec = uint32(ns);
    % 1. Configurar NOMBRES (coinciden con URDF)
    strArrayToAssign = {'cart_slider', 'pole_joint'};
    positionsToAssign = [q(1), q(2)];
    numStringsInArray = length(strArrayToAssign);
    msg.name_SL_Info.CurrentLength = uint32(numStringsInArray);
    for idx=1:numStringsInArray
        str = strArrayToAssign{idx};
        strLength = length(str);
        msg.name(idx).data(1:strLength) = uint8(str);
        msg.name(idx).data_SL_Info.CurrentLength = uint32(
            strLength);
    end
    % 2. POSICIONES (float64 array)
    numPositions = length(positionsToAssign);
    msg.position_SL_Info.CurrentLength = uint32(numPositions);
    msg.position(1:numPositions) = positionsToAssign;
    % 3. VELOCIDAD y ESFUERZO (vacíos)
    msg.velocity_SL_Info.CurrentLength = uint32(0);
    msg.effort_SL_Info.CurrentLength = uint32(0);
end
```

Esta función se encarga de mapear la información matemática simulada hacia un mensaje de tipo *sensor_msgs/JointState*, permitiendo que el visualizador gráfico de ROS (RViz) interprete la telemetría en tiempo real de acuerdo a las articulaciones definidas en el archivo URDF del robot.

Entradas: Estructura base vacía del mensaje de ROS (*blankMsg*), vector de posiciones generalizadas $q = [x, \theta]^T$, tiempo acumulado de la simulación (*time*) y la marca de tiempo absoluta en formato Unix Epoch obtenida al iniciar el proceso (*start_time_epoch*).

Salidas: Mensaje estructurado *msg* (*sensor_msgs/JointState*) listo para ser publicado en ROS.

Operación: El algoritmo procesa los datos siguiendo el protocolo estricto de mensajería de ROS:

1. **Sincronización Temporal:** Combina el tiempo base Unix con el reloj local del solucionador de Simulink, descomponiendo el tiempo total en segundos enteros (*sec*) y nanosegundos fraccionarios (*nanosec*) para rellenar el campo *header.stamp*.
2. **Asignación de Identificadores (Names):** Configura un arreglo de caracteres con los nombres de las articulaciones virtuales descritas en el URDF (*'cart_slider'* y *'pole_joint'*).
3. **Inyección de Telemetría (Positions):** Almacena de forma indexada el vector de estados continuos *q* dentro del arreglo de tipo *float64* reservado para las posiciones.
4. **Vaciado de Campos Auxiliares:** Configura explícitamente a cero la longitud de los arreglos de velocidad y esfuerzo (*velocity* y *effort*), solo se requiere el envío de datos posicionales para la sincronización de la cinemática directa en RViz.

En la Figura 6.1 se detalla la parametrización del *solver* numérico. Se ha seleccionado un paso de tiempo fijo (*Fixed-step*) con el algoritmo de integración *ode14x*, estableciendo un tamaño de paso de $\Delta t = 0.01$ s (10 ms).

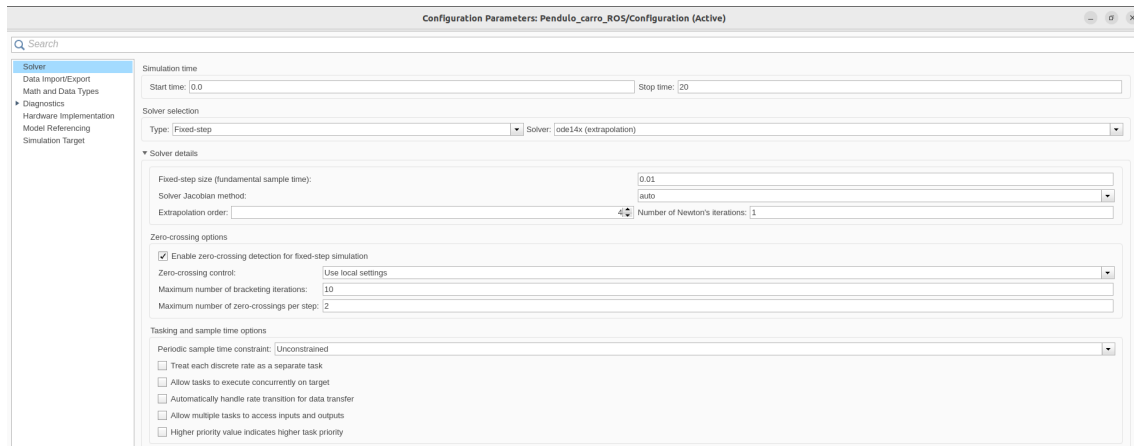


Figura 6.1: Configuración del *solver* con paso de tiempo fijo

Por otro lado, como se ilustra en la Figura 6.2, dentro del menú *Hardware Implementation* se ha establecido como objetivo la plataforma *Robot Operating System 2 (ROS 2)*.

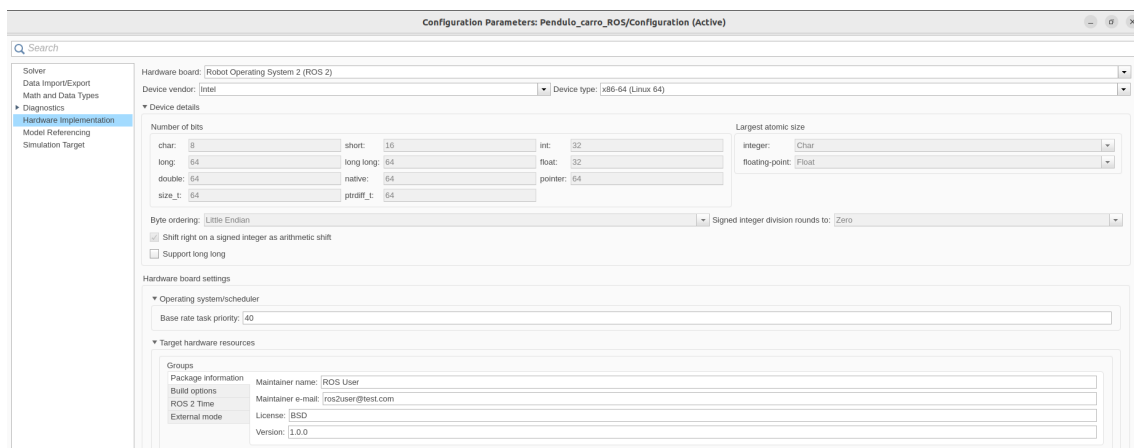


Figura 6.2: *Hardware Implementation*

E Desarrollo Dinámico y Programación del Bucle de Simulación

Para conectar la teoría matemática con la implementación en código, se parte de la definición $\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), u(t))$. Al aproximar la derivada temporal mediante el método de Euler de primer orden para un periodo de muestreo constante T , se obtiene la siguiente relación discreta:

$$\dot{\mathbf{x}}_k \approx \frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{T} \implies \mathbf{x}_{k+1} = \mathbf{x}_k + T \cdot f(\mathbf{x}_k, u_k) \quad (6.1)$$

Donde el vector de derivadas temporales $\mathbf{f} = [f_1, f_2, f_3, f_4]^T$ representa las velocidades y aceleraciones instantáneas del sistema en cada iteración k . Las velocidades se obtienen de forma directa como:

$$f_1 = \dot{x}_k = x_3 \quad (6.2)$$

$$f_2 = \dot{\theta}_k = x_4 \quad (6.3)$$

Sin embargo, para obtener las aceleraciones ($f_3 = \ddot{x}_k$, $f_4 = \ddot{\theta}_k$), se debe aislar el vector de aceleraciones generalizadas del modelo dinámico:

$$\mathbf{M}(\mathbf{q}_k) \begin{bmatrix} \ddot{x}_k \\ \ddot{\theta}_k \end{bmatrix} + \mathbf{C}(\mathbf{q}_k, \dot{\mathbf{q}}_k) \dot{\mathbf{q}}_k + \mathbf{G}(\mathbf{q}_k) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} u_k \quad (6.4)$$

$$\begin{bmatrix} f_3 \\ f_4 \end{bmatrix} = \begin{bmatrix} \ddot{x}_k \\ \ddot{\theta}_k \end{bmatrix} = \mathbf{M}(\mathbf{q}_k)^{-1} \left(\begin{bmatrix} u_k \\ 0 \end{bmatrix} - \mathbf{C}(\mathbf{q}_k, \dot{\mathbf{q}}_k) \dot{\mathbf{q}}_k - \mathbf{G}(\mathbf{q}_k) \right) \quad (6.5)$$

La traducción de este modelo matemático al entorno algorítmico se realiza estructurando las ecuaciones anteriores dentro de un bucle `for`. El código fuente desarrollado se presenta a continuación:

Implementación en MATLAB

```
for k=1:N-1
    % --- 1. CALCULO DE LA ACCION DE CONTROL ---
    u(k) = control(x(:,k), ref);

    % --- 2. DINAMICA (NO LINEAL) DEL SISTEMA ---
    M_t = [M,          +mp*lcm*cos(x(2,k));
           +mp*lcm*cos(x(2,k)),      J];

    C_t = [cc,      -mp*lcm*x(4,k)*sin(x(2,k))
           0,          cp          ];

    G_t = [0; +mp*g*lcm*sin(x(2,k))];

    % --- 3. SIMULACION DE LA SIGUIENTE ITERACION ---
    f1=x(3,k);
    f2=x(4,k);
    f34=M_t \ ([u(k);0]-C_t*x(3:4,k)-G_t);

    x(:,k+1)=x(:,k)+T*[f1;f2;f34];
end
```

El script completo de la simulación, junto con los parámetros reales del sistema y los controladores funcionales detallados, se encuentra disponible para su consulta y descarga en el repositorio del proyecto [23].

F Tabla de Multiplexación Completa del TMS32F280049C

0, 4, 8, 12	1	2	3	5	6	7	9	10	11	13	14	15
GPIO0	EPWM1_A				I2CA_SDA							
GPIO1	EPWM1_B				I2CA_SCL							
GPIO2	EPWM2_A			OUTPUT XBAR1	PMBUSA_ SDA		SCIA_TX	FSIRXA_D1				
GPIO3	EPWM2_B	OUTPUT XBAR2		OUTPUT XBAR2	PMBUSA_ SCL	SPIA_CLK	SCIA_RX	FSIRXA_D0				
GPIO4	EPWM3_A			OUTPUT XBAR3	CANA_TX			FSIRXA_ CLK				
GPIO5	EPWM3_B		OUTPUT XBAR3		CANA_RX	SPIA_STE	FSITXA_D1					
GPIO6	EPWM4_A	OUTPUT XBAR4	SYNCOUT	EQEP1_A	CANB_TX	SPIB_SOMI	FSITXA_D0					
GPIO7	EPWM4_B		OUTPUT XBAR5	EQEP1_B	CANB_RX	SPIB_SIMO	FSITXA_ CLK					
GPIO8	EPWM5_A	CANB_TX	ADCSOCAO	EQEP1_ STROBE	SCIA_TX	SPIA_SIMO	I2CA_SCL	FSITXA_D1				
GPIO9	EPWM5_B	SCIB_TX	OUTPUT XBAR6	EQEP1_ INDEX	SCIA_RX	SPIA_CLK		FSITXA_D0				
GPIO10	EPWM6_A	CANB_RX	ADCSOCBO	EQEP1_A	SCIB_TX	SPIA_SOMI	I2CA_SDA	FSITXA_ CLK				
GPIO11	EPWM6_B	SCIB_RX	OUTPUT XBAR7	EQEP1_B	SCIB_RX	SPIA_STE	FSIRXA_D1					
GPIO12	EPWM7_A	CANB_TX		EQEP1_ STROBE	SCIB_TX	PMBUSA_ CTL	FSIRXA_D0					
GPIO13	EPWM7_B	CANB_RX		EQEP1_ INDEX	SCIB_RX	PMBUSA_ ALERT	FSIRXA_ CLK					
GPIO14	EPWM8_A	SCIB_TX			OUTPUT XBAR3	PMBUSA_ SDA	SPIB_CLK	EQEP2_A				
GPIO15	EPWM8_B	SCIB_RX			OUTPUT XBAR4	PMBUSA_ SCL	SPIB_STE	EQEP2_B				

Figura 6.3: *GPIO Muxed Pins*, Parte I: GPIO0 a GPIO15 [22]

0, 4, 8, 12	1	2	3	5	6	7	9	10	11	13	14	15
GPIO16	SPIA_SIMO	CANB_TX	OUTPUT XBAR7	EPWM5_A	SCIA_TX	SD1_D1	EQEP1_ STROBE	PMBUSA_ SCL	XCLKOUT			
GPIO17	SPIA_SOMI	CANB_RX	OUTPUT XBAR8	EPWM5_B	SCIA_RX	SD1_C1	EQEP1_ INDEX	PMBUSA_ SDA				
GPIO18_X2	SPIA_CLK	SCIB_TX	CANA_RX	EPWM6_A	I2CA_SCL	SD1_D2	EQEP2_A	PMBUSA_ CTL	XCLKOUT			
GPIO20												
GPIO21												
GPIO22_VFBSW	EQEP1_ STROBE		SCIB_TX		SPIB_CLK	SD1_D4	LINA_TX					
GPIO23_VSW												
GPIO24	OUTPUT XBAR1	EQEP2_A		EPWM8_A	SPIB_SIMO	SD1_D1		PMBUSA_ SCL	SCIA_TX	ERRORSTS		
GPIO25	OUTPUT XBAR2	EQEP2_B			SPIB_SOMI	SD1_C1	FSITXA_D1	PMBUSA_ SDA	SCIA_RX			
GPIO26	OUTPUT XBAR3	EQEP2_ INDEX		OUTPUT XBAR3	SPIB_CLK	SD1_D2	FSITXA_D0	PMBUSA_ CTL	I2CA_SDA			
GPIO27	OUTPUT XBAR4	EQEP2_ STROBE		OUTPUT XBAR4	SPIB_STE	SD1_C2	FSITXA_ CLK	PMBUSA_ ALERT	I2CA_SCL			
GPIO28	SCIA_RX		EPWM7_A	OUTPUT XBAR5	EQEP1_A	SD1_D3	EQEP2_ STROBE	LINA_TX	SPIB_CLK	ERRORSTS		
GPIO29	SCIA_TX		EPWM7_B	OUTPUT XBAR6	EQEP1_B	SD1_C3	EQEP2_ INDEX	LINA_RX	SPIB_STE	ERRORSTS		
GPIO30	CANA_RX		SPIB_SIMO	OUTPUT XBAR7	EQEP1_ STROBE	SD1_D4						
GPIO31	CANA_TX		SPIB_SOMI	OUTPUT XBAR8	EQEP1_ INDEX	SD1_C4	FSIRXA_D1					
GPIO32	I2CA_SDA		SPIB_CLK	EPWM8_B	LINA_TX	SD1_D3	FSIRXA_D0	CANA_TX				
GPIO33	I2CA_SCL		SPIB_STE	OUTPUT XBAR4	LINA_RX	SD1_C3	FSIRXA_ CLK	CANA_RX				
GPIO34	OUTPUT XBAR1				PMBUSA_ SDA							
GPIO35	SCIA_RX		I2CA_SDA	CANA_RX	PMBUSA_ SCL	LINA_RX	EQEP1_A	PMBUSA_ CTL				TDI
GPIO37	OUTPUT XBAR2		I2CA_SCL	SCIA_TX	CANA_TX	LINA_TX	EQEP1_B	PMBUSA_ ALERT				TDO

Figura 6.3: *GPIO Muxed Pins*, Parte II: GPIO16 a GPIO37 (continuación) [22]

0, 4, 8, 12	1	2	3	5	6	7	9	10	11	13	14	15
GPIO39					CANB_RX	FSIRXA_CLK						
GPIO40					PMBUSA_SDA	FSIRXA_D0	SCIB_TX	EQEP1_A				
GPIO41												
GPIO42												
GPIO43												
GPIO44												
GPIO45												
GPIO46												
GPIO47												
GPIO48												
GPIO49												
GPIO50												
GPIO51												
GPIO52												
GPIO53												
GPIO54												
GPIO55												
GPIO56	SPIA_CLK			EQEP2_STROBE	SCIB_TX	SD1_D3	SPIB_SIMO		EQEP1_A			
GPIO57	SPIA_STE			EQEP2_INDEX	SCIB_RX	SD1_C3	SPIB_SOMI		EQEP1_B			
GPIO58				OUTPUT_XBAR1	SPIB_CLK	SD1_D4	LINA_TX	CANB_TX	EQEP1_STROBE			
GPIO59				OUTPUT_XBAR2	SPIB_STE	SD1_C4	LINA_RX	CANB_RX	EQEP1_INDEX			

Figura 6.3: *GPIO Muxed Pins*, Parte III: GPIO39 a GPIO59 (continuación) [22]

G Registros del módulo eQEP

En primer lugar, se proporciona el mapa de memoria global del módulo de hardware para la lectura de encoders (*eQEP*), el cual delimita las direcciones base del periférico y sus registros de control asociados.

Offset	Acronym	Register Name	Write Protection	Section
0h	QPOSCNT	Position Counter		Go
2h	QPOSINIT	Position Counter Init		Go
4h	QPOSMAX	Maximum Position Count		Go
6h	QPOSCMP	Position Compare		Go
8h	QPOSILAT	Index Position Latch		Go
Ah	QPOSSLAT	Strobe Position Latch		Go
Ch	QPOSLAT	Position Latch		Go
Eh	QUTMR	QEP Unit Timer		Go
10h	QUPRD	QEP Unit Period		Go
12h	QWDTMR	QEP Watchdog Timer		Go
13h	QWDPRD	QEP Watchdog Period		Go
14h	QDECCTL	Quadrature Decoder Control		Go
15h	QEPCTL	QEP Control		Go
16h	QCAPCTL	Quadrature Capture Control		Go
17h	QPOSCTL	Position Compare Control		Go
18h	QEINT	QEP Interrupt Control		Go
19h	QFLG	QEP Interrupt Flag		Go
1Ah	QCLR	QEP Interrupt Clear		Go
1Bh	QFRC	QEP Interrupt Force		Go
1Ch	QEPSTS	QEP Status		Go
1Dh	QCTMR	QEP Capture Timer		Go
1Eh	QCPRD	QEP Capture Period		Go
1Fh	QCTMRLAT	QEP Capture Latch		Go
20h	QCPRLAT	QEP Capture Period Latch		Go
30h	REV	QEP Revision Number		Go
32h	QEPSTROBESEL	QEP Strobe select register		Go
34h	QMACTRL	QMA Control register		Go

Figura 6.4: Mapa de memoria del periférico *eQEP* [22]

Debido a la gran densidad de funciones integradas en el periférico y con el propósito de optimizar la lectura técnica de este documento, en las secciones siguientes se desglosa la estructura interna de bits únicamente de aquellos registros de control modificados de forma explícita en la rutina de inicialización. Este análisis selectivo justifica el impacto funcional de cada campo binario configurado sobre el comportamiento dinámico de la planta.

El primer bloque crítico modificado en el código fuente corresponde al registro *QDECCTL*. En la Figura 6.6 se detalla la distribución y propósito de sus campos binarios según la documentación oficial del fabricante.

15	14	13	12	11	10	9	8
QSRC	SOEN	SPSEL	XCR	SWAP	IGATE	QAP	
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
QBP	QIP	QSP	RESERVED				
R/W-0h	R/W-0h	R/W-0h	R-0h				

Figura 6.5: Registro *QDECCTL* [22].

Bit	Field	Type	Reset	Description
15-14	QSRC	R/W	0h	Position-counter source selection Reset type: SYSRSn 0h (R/W) = Quadrature count mode (QCLK = iCLK, QDIR = iDIR) 1h (R/W) = Direction-count mode (QCLK = xCLK, QDIR = xDIR) 2h (R/W) = UP count mode for frequency measurement (QCLK = xCLK, QDIR = 1) 3h (R/W) = DOWN count mode for frequency measurement (QCLK = xCLK, QDIR = 0)
13	SOEN	R/W	0h	Sync output-enable Reset type: SYSRSn 0h (R/W) = Disable position-compare sync output 1h (R/W) = Enable position-compare sync output
12	SPSEL	R/W	0h	Sync output pin selection Reset type: SYSRSn 0h (R/W) = Index pin is used for sync output 1h (R/W) = Strobe pin is used for sync output
11	XCR	R/W	0h	External Clock Rate Reset type: SYSRSn 0h (R/W) = 2x resolution: Count the rising/falling edge 1h (R/W) = 1x resolution: Count the rising edge only
10	SWAP	R/W	0h	CLK/DIR Signal Source for Position Counter Reset type: SYSRSn 0h (R/W) = Quadrature-clock inputs are not swapped 1h (R/W) = Quadrature-clock inputs are swapped
9	IGATE	R/W	0h	Index pulse gating option Reset type: SYSRSn 0h (R/W) = Disable gating of Index pulse 1h (R/W) = Gate the index pin with strobe
8	QAP	R/W	0h	QEPA input polarity Reset type: SYSRSn 0h (R/W) = No effect 1h (R/W) = Negates QEPA input
7	QBP	R/W	0h	QEPB input polarity Reset type: SYSRSn 0h (R/W) = No effect 1h (R/W) = Negates QEPB input
6	QIP	R/W	0h	QEPI input polarity Reset type: SYSRSn 0h (R/W) = No effect 1h (R/W) = Negates QEPI input

Figura 6.6: Distribución de campos y bits de configuración del registro QDECCTL [22].

Bit	Field	Type	Reset	Description
5	QSP	R/W	0h	QEPS input polarity Reset type: SYSRSn 0h (R/W) = No effect 1h (R/W) = Negates QEPS input
4-0	RESERVED	R	0h	Reserved

Figura 6.6: Distribución de campos y bits de configuración del registro QDECCTL [22] (continuación).

A partir de la distribución expuesta en la Figura 6.6, las asignaciones lógicas realizadas en el código se justifican técnicamente de la siguiente manera:

- **QDECCTL.bit.QSRC = 0h:** Configura el modo de conteo en cuadratura pura (*Quadrature count mode*).
- **QDECCTL.bit.SWAP = 0h:** Determina que las entradas de reloj y dirección no se intercambian por hardware (*Not swapped*). Esto asegura la concordancia directa entre el sentido de giro real del eje y el incremento del contador de posición.

Finalmente, para el registro QEPCTL se detalla la distribución y propósito de sus campos binarios según la documentación oficial del fabricante.

15	14	13	12	11	10	9	8
FREE_SOFT		PCRM		SEI		IEI	
R/W-0h		R/W-0h		R/W-0h		R/W-0h	
7	6	5	4	3	2	1	0
SWI	SEL	IEL		QPEN	QCLM	UTE	WDE
R/W-0h	R/W-0h	R/W-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h

Figura 6.7: Registro QEPCTL [22].

Bit	Field	Type	Reset	Description
15-14	FREE_SOFT	R/W	0h	Emulation mode Reset type: SYSRSn 0h (R/W) = QPOSCNT behavior Position counter stops immediately on emulation suspend 0h (R/W) = QWDTMR behavior Watchdog counter stops immediately 0h (R/W) = QUTMR behavior Unit timer stops immediately 0h (R/W) = QCTMR behavior Capture Timer stops immediately 1h (R/W) = QPOSCNT behavior Position counter continues to count until the rollover 1h (R/W) = QWDTMR behavior Watchdog counter counts until WD period match roll over 1h (R/W) = QUTMR behavior Unit timer counts until period rollover 1h (R/W) = QCTMR behavior Capture Timer counts until next unit period event 2h (R/W) = QPOSCNT behavior Position counter is unaffected by emulation suspend 2h (R/W) = QWDTMR behavior Watchdog counter is unaffected by emulation suspend 2h (R/W) = QUTMR behavior Unit timer is unaffected by emulation suspend 2h (R/W) = QCTMR behavior Capture Timer is unaffected by emulation suspend 3h (R/W) = Same as FREE_SOFT_2
13-12	PCRM	R/W	0h	Position counter reset Reset type: SYSRSn 0h (R/W) = Position counter reset on an index event 1h (R/W) = Position counter reset on the maximum position 2h (R/W) = Position counter reset on the first index event 3h (R/W) = Position counter reset on a unit time event
11-10	SEI	R/W	0h	Strobe event initialization of position counter Reset type: SYSRSn 0h (R/W) = Does nothing (action disabled) 1h (R/W) = Does nothing (action disabled) 2h (R/W) = Initializes the position counter on rising edge of the QEPS signal 3h (R/W) = Clockwise Direction: Initializes the position counter on the rising edge of QEPS strobe Counter Clockwise Direction: Initializes the position counter on the falling edge of QEPS strobe

Figura 6.8: Distribución de campos y bits de configuración del registro QEPCTL [22].

Bit	Field	Type	Reset	Description
9-8	IEI	R/W	0h	Index event init of position count Reset type: SYSRSn 0h (R/W) = Do nothing (action disabled) 1h (R/W) = Do nothing (action disabled) 2h (R/W) = Initializes the position counter on the rising edge of the QEPI signal (QPOSCNT = QPOSINIT) 3h (R/W) = Initializes the position counter on the falling edge of QEPI signal (QPOSCNT = QPOSINIT)
7	SWI	R/W	0h	Software init position counter Reset type: SYSRSn 0h (R/W) = Do nothing (action disabled) 1h (R/W) = Initialize position counter (QPOSCNT=QPOSINIT). This bit is not cleared automatically
6	SEL	R/W	0h	Strobe event latch of position counter Reset type: SYSRSn 0h (R/W) = The position counter is latched on the rising edge of QEPS strobe (QPOSSLAT = POSCCNT). Latching on the falling edge can be done by inverting the strobe input using the QSP bit in the QDECCTL register 1h (R/W) = Clockwise Direction: Position counter is latched on rising edge of QEPS strobe Counter Clockwise Direction: Position counter is latched on falling edge of QEPS strobe
5-4	IEL	R/W	0h	Index event latch of position counter (software index marker) Reset type: SYSRSn 0h (R/W) = Reserved 1h (R/W) = Latches position counter on rising edge of the index signal 2h (R/W) = Latches position counter on falling edge of the index signal 3h (R/W) = Software index marker. Latches the position counter and quadrature direction flag on index event marker. The position counter is latched to the QPOSILAT register and the direction flag is latched in the QEPSTS[QDLF] bit. This mode is useful for software index marking.
3	QPEN	R/W	0h	Quadrature position counter enable/software reset Reset type: SYSRSn 0h (R/W) = Reset the eQEP peripheral internal operating flags/read-only registers. Control/configuration registers are not disturbed by a software reset. When QPEN is disabled, some flags in the QFLG register do not get reset or cleared and show the actual state of that flag. 1h (R/W) = eQEP position counter is enabled
2	QCLM	R/W	0h	QEP capture latch mode Reset type: SYSRSn 0h (R/W) = Latch on position counter read by CPU. Capture timer and capture period values are latched into QCTMRLAT and QCPRDLAT registers when CPU reads the QPOSCNT register. 1h (R/W) = Latch on unit time out. Position counter, capture timer and capture period values are latched into QPOSILAT, QCTMRLAT and QCPRDLAT registers on unit time out.
1	UTE	R/W	0h	QEP unit timer enable Reset type: SYSRSn 0h (R/W) = Disable eQEP unit timer 1h (R/W) = Enable unit timer
0	WDE	R/W	0h	QEP watchdog enable Reset type: SYSRSn 0h (R/W) = Disable the eQEP watchdog timer 1h (R/W) = Enable the eQEP watchdog timer

Figura 6.8: Distribución de campos y bits de configuración del registro QEPCTL [22] (continuación).

A partir de la distribución expuesta en la Figura 6.8, las configuraciones aplicadas en el firmware se justifican técnicamente de la siguiente manera:

- **QEPCTL.bit.FREE_SOFT = 2h:** Impide que el contador de posición se detenga debido a un punto de ruptura en la depuración.
- **QEPCTL.bit.QPEN = 1h:** Habilita formalmente el contador de posición del eQEP (*eQEP Position Counter Enable*).

H Registros del módulo ePWM